

Aula 9 - Cifras de bloco

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
June 2, 2026

Cifras de Fluxo vs Cifras de Bloco

Cifras de Fluxo:

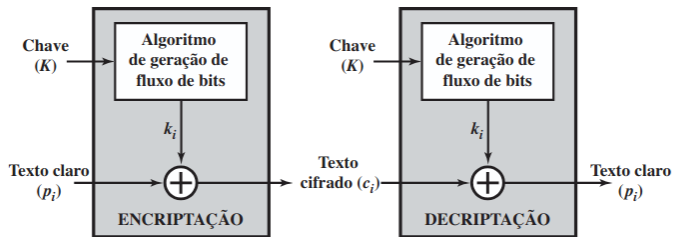
- Encriptam dados um **bit** ou **byte** por vez.
- Exemplos clássicos: Vigenère autochaveada e Vernam.
- Ideal: One-time pad (inquebrável se o fluxo de chaves for verdadeiramente aleatório).
- Limitação prática: o fluxo de chaves precisa ser compartilhado previamente via canal seguro.
- Solução prática: gerar o fluxo de bits algoritmicamente, controlado por chave, garantindo que partes futuras do fluxo sejam imprevisíveis.

Cifras de Bloco:

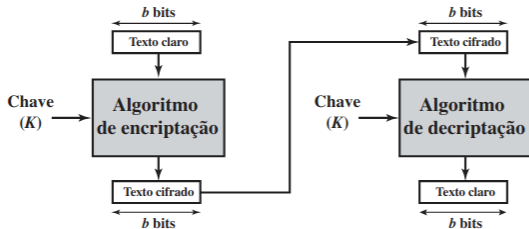
- Tratam um **bloco** de texto claro como um todo, normalmente de 64 ou 128 bits.
- Usuários compartilham uma chave simétrica.
- Modos de operação permitem uso semelhante ao das cifras de fluxo.
- Mais analisadas e amplamente utilizadas em aplicações de criptografia simétrica.

Cifra de fluxo vs. Cifra de bloco

Figura 3.1 Cifra de fluxo e cifra de bloco.



(a) Cifra de fluxo usando gerador algorítmico de fluxo de bits



Confusão e Difusão segundo Claude Shannon

Contexto histórico:

- Claude Shannon desenvolveu a ideia de **cifras de produto**, alternando funções de confusão e difusão [?].
- A estrutura da **cifra de Feistel**, baseada na proposta de Shannon de 1945, é utilizada em muitas cifras de bloco simétricas atuais.

Conceitos de Shannon:

- **Difusão**: espalhar a influência de cada bit do texto claro por muitos bits do texto cifrado, dificultando deduções estatísticas.
- **Confusão**: tornar a relação entre chave e texto cifrado complexa, de modo que conhecer parte da saída não revele informações sobre a chave.

Objetivo: impedir criptoanálise baseada em estatísticas do texto claro, como frequências de letras ou palavras prováveis.

Difusão em Criptografia

A difusão busca dissociar a estrutura estatística do texto claro das estatísticas do texto cifrado.

Princípio: Cada dígito do texto claro deve afetar muitos dígitos do texto cifrado, espalhando a informação.

Exemplo matemático: Para uma mensagem $M = m_1, m_2, m_3, \dots$:

$$y_n = \left(\sum_{i=1}^k m_{n+i} \right) \bmod 26$$

onde k letras sucessivas do texto claro influenciam cada letra do texto cifrado.

Efeito:

- Frequências de letras e dígrafos no texto cifrado se aproximam de uma distribuição uniforme.
- Em cifras de bloco binárias, difusão é alcançada por permutações seguidas de funções de transformação, garantindo que bits de diferentes posições contribuam para cada bit cifrado.

Difusão e Confusão em Cifras de Bloco

Cada cifra de bloco transforma um bloco de texto claro em texto cifrado, dependendo da chave.

Difusão:

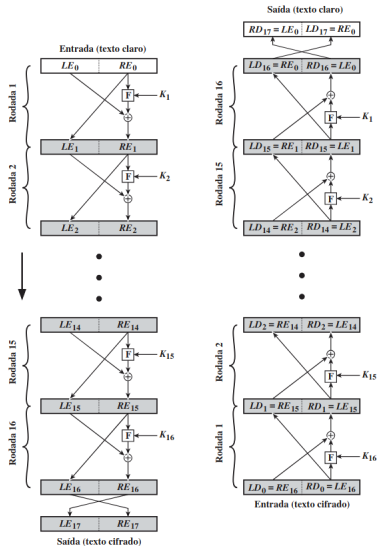
- Complica o relacionamento estatístico entre o texto claro e o texto cifrado.
- Faz com que cada bit do texto claro influencie muitos bits do texto cifrado.
- Frustra tentativas de deduzir padrões no texto claro.

Confusão:

- Complica o relacionamento entre o texto cifrado e a chave de encriptação.
- Utiliza funções de substituição complexas para dificultar a descoberta da chave.
- Funções lineares simples resultariam em pouca confusão e vulnerabilidade.

Cifra de Feistel

Figura 3.3 Encriptação e decifração de Feistel (16 rodadas).



Estrutura da Cifra de Feistel

A estrutura de Feistel permite criar cifras de bloco seguras a partir de funções de encriptação simples.

Descrição:

- Entrada: bloco de texto claro de $2w$ bits e chave K .
- O bloco é dividido em duas metades: L_0 e R_0 .
- Os dados passam por n rodadas de processamento.
- Cada rodada i recebe L_{i-1} , R_{i-1} e uma subchave K_i derivada de K .
- As subchaves K_i são normalmente diferentes entre si e da chave original.
- Após n rodadas, as metades são combinadas para gerar o bloco cifrado.

Exemplo: 16 rodadas, mas qualquer número de rodadas pode ser implementado.

Rodadas da Cifra de Feistel

Todas as rodadas têm a mesma estrutura básica:

Processamento em cada rodada:

1. Aplica-se uma função de substituição F à metade direita dos dados R_{i-1} , parametrizada pela subchave K_i .
2. Realiza-se a operação XOR entre a saída de F e a metade esquerda L_{i-1} :

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

3. Executa-se a permutação trocando as metades:

$$L_i = R_{i-1}$$

Observações:

- F é uma função de w bits de R_{i-1} e y bits de K_i , produzindo w bits de saída.
- Estrutura geral é repetida em todas as rodadas, formando uma rede de substituição-permutação (SPN) como proposta por Shannon.

Fatores que influenciam a segurança da cifra de Feistel

A execução de uma rede de Feistel depende de diversos parâmetros de projeto, sendo um dos principais:

Tamanho de bloco:

- Blocos maiores proporcionam maior segurança, mantendo os demais fatores constantes.
- Blocos maiores reduzem a velocidade de encriptação/decriptação para um dado algoritmo.
- Maior segurança é obtida por meio de maior difusão.
- Tradicionalmente, o tamanho de bloco de 64 bits era considerado adequado para cifras de bloco.
- O AES moderno utiliza tamanho de bloco de 128 bits.

Fatores que influenciam a segurança da cifra de Feistel

- **Tamanho da chave:**

- Chave maior $\Rightarrow$ maior resistência a ataques de força bruta e maior confusão.
- Impacto: pode reduzir a velocidade de encriptação/decriptação.
- 64 bits ou menos: inseguros.
- 128 bits: tornou-se padrão comum.

- **Número de rodadas:**

- 1 rodada: segurança inadequada.
- 16 rodadas: valor típico que garante segurança aceitável.

- **Algoritmo de geração de subchaves:**

- Quanto mais complexo, mais difícil a criptoanálise.

- **Função F:**

- Função mais complexa $\Rightarrow$ maior resistência a ataques.

Considerações adicionais no projeto da cifra de Feistel

- **Encriptação/Decriptação rápidas em software:**
 - Muitas vezes a implementação ocorre em software, não em hardware.
 - Desempenho do algoritmo passa a ser um fator crítico.
- **Facilidade de análise:**
 - Algoritmos claros e concisos são mais fáceis de avaliar contra ataques.
 - Transparência aumenta a confiança na robustez criptográfica.
 - Exemplo: o DES não possui estrutura de fácil análise.

Algoritmo de Decriptação de Feistel

O processo de decriptação com uma cifra de Feistel é basicamente o mesmo da encriptação. A regra é a seguinte: use o texto cifrado como entrada para o algoritmo, mas aplique as subchaves K_i em ordem reversa.

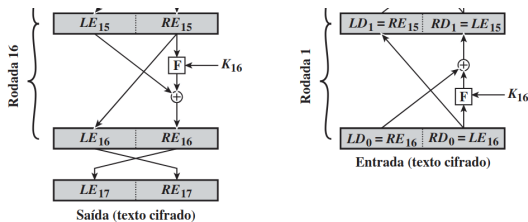
Ou seja:

- Use K_n na primeira rodada, K_{n-1} na segunda, $\dots$, até K_1 na última rodada.

Observação: Isso permite que o mesmo algoritmo seja usado tanto para encriptação quanto para decriptação.

Validação do Algoritmo de Feistel com Ordem de Chaves Invertida

Observando a última rodada de encriptação da Cifra de Feistel e a primeira rodada da deciptação



E sabendo que a operação lógica ou-exclusivo tem as seguintes propriedades:

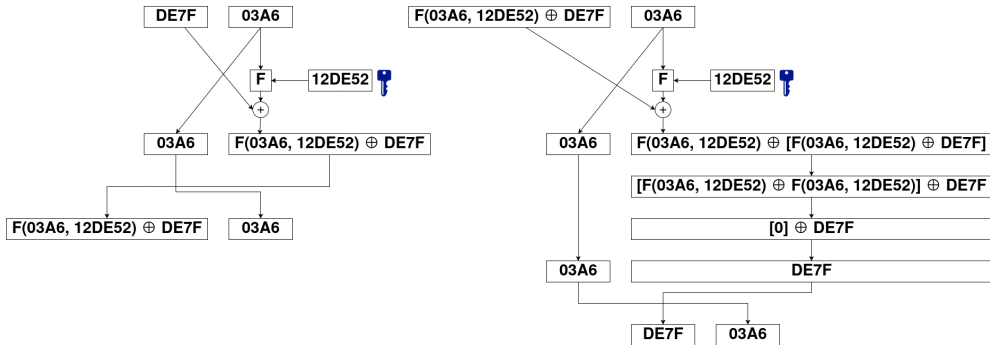
$$[A \oplus B] \oplus C = A \oplus [B \oplus C]$$

$$D \oplus D = 0$$

$$E \oplus 0 = E$$

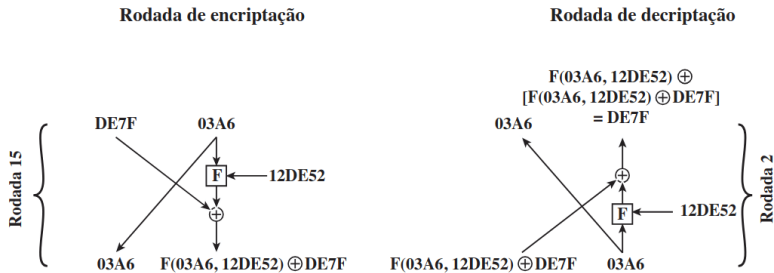
Validação do Algoritmo de Feistel com Ordem de Chaves Invertida

Temos:



Temos:

Figura 3.4 Exemplo Feistel.



Data Encryption Standard (DES)

Até a introdução do **Advanced Encryption Standard (AES)** em 2001, o DES era o esquema de encriptação mais utilizado.

Histórico:

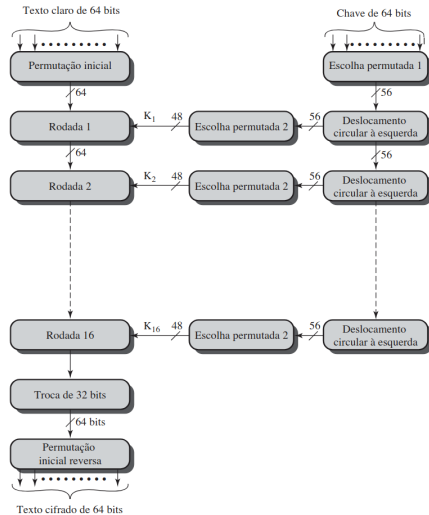
- Adotado em 1977 pelo **NIST**.
- Também conhecido como **Data Encryption Algorithm (DEA)**.
- Encripta dados em blocos de 64 bits usando uma chave de 56 bits.
- O mesmo algoritmo e chave são usados para encriptação e deciptação.

Evolução:

- Em 1994, o NIST reafirmou o DES para uso federal, recomendando restrição a informações não confidenciais.
- Em 1999, o NIST indicou que o DES seria apenas para sistemas legados e recomendou o **Triple DES**.
 - Triple DES repete o algoritmo DES três vezes usando duas ou três chaves diferentes.
- Hoje recomendamos o uso do **AES**

Esquema do DES

Figura 3.5 Representação geral do algoritmo de encriptação DES.



No esquema de encriptação DES existem duas entradas na função: **texto claro** (64 bits) e **chave** (56 bits).

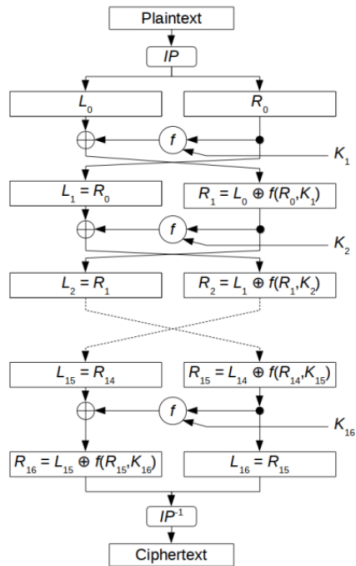
Processamento do texto claro:

- **Permutação Inicial (IP)**: reorganiza os bits do texto claro para produzir a entrada permutada.
- **16 rodadas de função Feistel**: cada rodada envolve permutação e substituição dos bits.
- **Troca das metades esquerda e direita** para gerar a pré-saída.
- **Permutação Final (IP^{-1})**: inverso da permutação inicial, produzindo o texto cifrado de 64 bits.

Data Encryption Standard (DES)

- Cifra de bloco que processa dados em blocos de 64 bits.
- Entrada: bloco de texto claro de 64 bits.
- Saída: bloco de texto cifrado de 64 bits.
- Algoritmo simétrico: mesma chave e mesmo algoritmo para encriptação e deciptação (com pequenas diferenças no escalonamento de chaves).

Esquema detalhado - DES



Chaves no DES

- Comprimento efetivo da chave: 56 bits.
- A chave é representada como um número de 64 bits:
 - A cada byte, o bit menos significativo é usado para verificação de paridade.
 - Esses 8 bits de paridade não participam da criptografia.
- Existem algumas chaves fracas, mas são facilmente evitáveis.
- Toda a segurança do DES depende da chave utilizada.

Processamento da chave:

- Chave de 56 bits passa por uma permutação inicial.
- Para cada uma das 16 rodadas, gera-se uma subchave (K_i) usando:
 - Deslocamento circular à esquerda.
 - Permutação fixa.
- Cada rodada utiliza uma subchave diferente, derivada da chave original.

Decriptação DES: Assim como qualquer cifra de Feistel, a decriptação usa o mesmo algoritmo da encriptação, exceto que a aplicação das subchaves é invertida. Além disso, as permutações inicial e final são invertidas.

Outline of the DES Algorithm

- O DES opera sobre blocos de **64 bits**.
- Etapas principais:
 - **Permutação inicial (IP)** sobre o bloco de entrada.
 - Divisão em duas metades:
 - Lado esquerdo (32 bits).
 - Lado direito (32 bits).

Rodadas e Finalização do DES

- São realizadas **16 rodadas** de operações idênticas.
- Em cada rodada:
 - A função f combina dados com subchaves derivadas da chave principal.
- Após a 16ª rodada:
 - As metades esquerda e direita são reunidas.
 - Aplica-se a **permutação final**, inversa da permutação inicial.

DES Round Function f

- Em cada rodada:
 - A chave de 56 bits é **deslocada** e **reduzida** para 48 bits.
 - O lado direito (32 bits) é **expandido** para 48 bits.
 - Os 48 bits resultantes são combinados com a subchave via **XOR**.
 - O resultado passa por **8 S-boxes**, produzindo 32 bits.
 - Esses 32 bits sofrem uma **permutação**.
- A saída da função f é combinada com o lado esquerdo via XOR.
- Após isso, ocorre a **troca de metades**.

Equações de uma Rodada no DES

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

- L_i, R_i : metades esquerda e direita no final da rodada i .
- K_i : subchave de 48 bits da rodada i .
- f : função que realiza expansão, XOR, substituições e permutação.
- Repetido por **16 rodadas**.

Uma rodada do DES

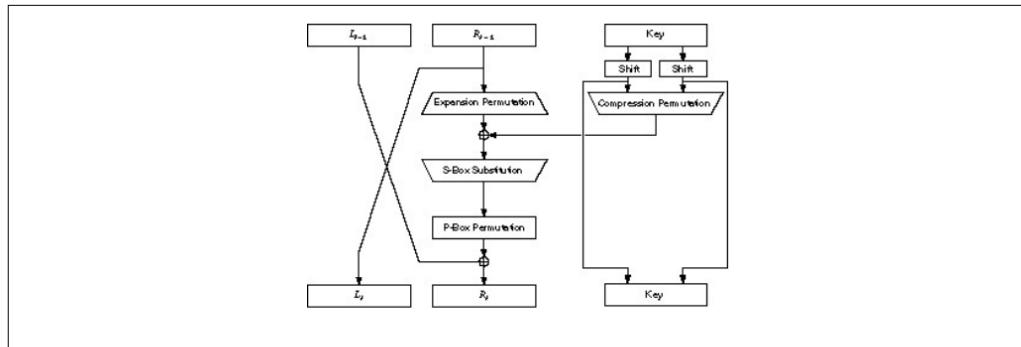


Figure 12.2 One round of DES.

Exemplo do DES

Exemplo prático do DES:

- **Objetivo:** observar os padrões em hexadecimal que surgem de uma etapa para outra, sem necessariamente replicar o cálculo manualmente
- Texto claro escolhido é um **palíndromo hexadecimal**.

Texto claro:	02468aceeca86420
Chave:	0f1571c947d9e859
Texto cifrado:	da02ce3a89ecac3b

Exemplo do DES

- Evolução do algoritmo ao longo das rodadas.
- Primeira linha: valores de 32 bits das metades esquerda (L) e direita (R) **após a permutação inicial (IP)**.
- Linhas seguintes (1 a 16): resultado após cada rodada, incluindo:
- Última linha: valores de L e R após a permutação inicial inversa (IP^{-1}).
- A concatenação desses dois valores gera o **texto cifrado**.

Tabela 3.2 Exemplo de DES.

Rodada	K_i	L_i	R_i
IP		5a005a00	3cf03c0f
1	1e030f03080d2930	3cf03c0f	bad22845
2	0a31293432242318	bad22845	99e9b723
3	23072318201d0c1d	99e9b723	0bae3b9e
4	05261d3824311a20	0bae3b9e	42415649
5	3325340136002c25	42415649	18b3fa41
6	123a2d0d04262a1c	18b3fa41	9616fe23
7	021f120b1c130611	9616fe23	67117cf2
8	1c10372a2832002b	67117cf2	c11bfc09

9	04292a380c341f03	c11bfc09	887fbc6c
10	2703212607280403	887fbc6c	600f7e8b
11	2826390c31261504	600f7e8b	f596506e
12	12071c241a0a0f08	f596506e	738538b8
13	300935393c0d100b	738538b8	c6a62c4e
14	311e09231321182a	c6a62c4e	56b0bd75
15	283d3e0227072528	56b0bd75	75e8fd8f
16	2921080b13143025	75e8fd8f	25896490
IP ⁻¹		da02ce3a	89ecac3b

Nota: as subchaves DES são apresentadas como oito valores de 6 bits no padrão hexa.

Efeito Avalanche

- Propriedade desejável em algoritmos de encriptação.
- Uma pequena mudança no texto claro ou na chave deve causar uma grande alteração no texto cifrado.
- **Objetivo:** Alterar apenas **um bit** no texto claro ou na chave deve modificar **muitos bits** no resultado.
- **Resultado esperado:** Caso a mudança fosse pequena, seria possível reduzir o espaço de busca de textos claros ou de chaves.
- O efeito avalanche aumenta a segurança, dificultando ataques de análise e padrões previsíveis.

Exemplo do Efeito Avalanche no DES

- Texto claro inicial modificado no 4º bit.
- Tabela de acompanhamento mostra:
 - Coluna 1: rodadas do algoritmo.
 - Coluna 2: valores intermediários de 64 bits no final de cada rodada.
 - Coluna 3: número de bits diferentes entre os dois textos.
- Após apenas 3 rodadas: diferença de **18 bits**.
- Texto cifrado final: diferença de **32 bits**.
- Demonstra claramente o **efeito avalanche** do DES.

Exemplo do Efeito Avalanche no DES

Tabela 3.3 Efeito avalanche no DES: mudança no texto claro.

Rodada		δ
	02468aceeca86420 12468aceeca86420	1
1	3cf03c0fbad22845 3cf03c0fbad32845	1
2	bad2284599e9b723 bad3284539a9b7a3	5
3	99e9b7230bae3b9e 39a9b7a3171cb8b3	18
4	0bae3b9e42415649 171cb8b3ccaca55e	34
5	4241564918b3fa41 ccaca55ed16c3653	37
6	18b3fa419616fe23 d16c3653cf402c68	33
7	9616fe2367117cf2 cf402c682b2cefbc	32
8	67117cf2c11bfc09 2b2cefbc99f91153	33

Rodada		δ
9	c11bfc09887fbc6c 99f911532eed7d94	32
10	887fbc6c600f7e8b 2eed7d94d0f23094	34
11	600f7e8bf596506e d0f23094455da9c4	37
12	f596506e738538b8 455da9c47f6e3cf3	31
13	738538b8c6a62c4e 7f6e3cf34bcla8d9	29
14	c6a62c4e56b0bd75 4bcla8d91e07d409	33
15	56b0bd7575e8fd8f 1e07d4091ce2e6dc	31
16	75e8fd8f25896490 1ce2e6dc365e5f59	32
IP-1	da02ce3a89ecac3b 057cde97d7683f2a	32

Efeito Avalanche com Alteração da Chave

Experimento: Texto claro mantido constante mas com duas chaves utilizadas:

- Chave original modificada **com diferença no 4º bit**

Resultados:

- Diferença significativa no texto cifrado final — cerca de metade dos bits são diferentes.
- O efeito avalanche já aparece após poucas rodadas do DES.

Demonstra: Robustez do algoritmo contra pequenas alterações na chave.

Exemplo do Efeito Avalanche no DES - Mudança na chave

Tabela 3.4 Efeito avalanche no DES: mudança na chave.

Rodada		δ
	02468aceeca86420 02468aceeca86420	0
1	3cf03c0fbad22845 3cf03c0f9ad628c5	3
2	bad2284599e9b723 9ad628c59939136b	11
3	99e9b7230bae3b9e 9939136b768067b7	25
4	0bae3b9e42415649 768067b75a8807c5	29
5	4241564918b3fa41 5a8807c5488dbe94	26
6	18b3fa419616fe23 488dbe94aba7fe53	26
7	9616fe2367117cf2 aba7fe53177d21e4	27
8	67117cf2c11bfc09 177d21e4548f1de4	32

Rodada		δ
9	c11bfc09887fbc6c 548f1de471f64dfd	34
10	887fbc6c600f7e8b 71f64dfd4279876c	36
11	600f7e8bf596506e 4279876c399fdc0d	32
12	f596506e738538b8 399fdc0d6d208dbb	28
13	738538b8c6a62c4e 6d208dbbb9bdeea	33
14	c6a62c4e56b0bd75 b9bdeeaad2c3a56f	30
15	56b0bd7575e8fd8f d2c3a56f2765c1fb	33
16	75e8fd8f25896490 2765c1fb01263dc4	30
IP ⁻¹	da02ce3a89ecac3b ee92b50606b62b0b	30

A Força do DES

- Desde sua adoção como padrão federal, surgiram preocupações sobre a segurança do DES.
- Principais áreas de preocupação:
 - **Tamanho da chave:** 56 bits pode ser vulnerável a ataques de força bruta.
 - **Natureza do algoritmo:** estrutura e procedimentos internos do DES podem influenciar sua resistência a ataques criptográficos.
- Essas questões motivaram o desenvolvimento de algoritmos mais seguros, como o Triple DES (3DES) e o AES.

Uso de Chaves de 56 Bits no DES

- Chave de 56 bits $\rightarrow 2^{56} \approx 7,2 \times 10^{16}$ chaves possíveis.
- Ataque de força bruta **pareceria** impraticável:
 - Pesquisando metade do espaço de chaves
 - Uma máquina realizando 1 encriptação por microssegundo levaria mais de 1000 anos.
- No entanto, Diffie e Hellman (1977) propuseram tecnologia paralela:
 - Máquina com 1 milhão de dispositivos, cada um realizando 1 encriptação/ μ s.
 - Tempo médio de busca: cerca de 10 horas.
 - Custo estimado: US\$ 20 milhões (em 1977).
- Demonstra que, apesar do tamanho da chave, o DES não é invulnerável a ataques especializados.

Impacto da Tecnologia Moderna na Segurança do DES

- Atualmente, nem é necessário hardware especial para ataques de força bruta contra o DES.
- Processadores comerciais modernos já oferecem velocidades que ameaçam a segurança do DES:
 - Computadores multicore atuais: 10^9 combinações de chaves por segundo [SEAG08].
 - Testes em máquinas Intel multicore: 5×10^8 encriptações por segundo [BASU12].
 - Supercomputadores contemporâneos: 10^{13} encriptações por segundo.
- Instruções de hardware recentes para AES também aceleram operações criptográficas, mostrando que ataques de força bruta se tornam cada vez mais viáveis.
- Conclusão: o DES, com chave de 56 bits, é vulnerável diante da tecnologia atual.

Força Bruta e Tamanho da Chave

- Tabela 3.5 mostra o tempo necessário para ataques de força bruta em diferentes tamanhos de chave.
- Exemplos:
 - Um único PC moderno pode quebrar o DES (56 bits) em 1 ano.
 - Vários PCs em paralelo reduzem drasticamente o tempo.
 - Supercomputadores contemporâneos: 1 hora para descobrir a chave do DES.
- Chaves de 128 bits ou mais são efetivamente inquebráveis por força bruta:
 - Mesmo com aceleração de 10^{12} vezes, ainda seriam necessários 100 mil anos.
- Alternativas mais seguras ao DES: **AES** e **Triple DES**

Exemplo do Efeito Avalanche no DES - Mudança na chave

Tabela 3.5 Tempo médio exigido para uma busca exaustiva no espaço de chaves.

Tamanho de chave (bits)	Cifra	Número de chaves alternativas	Tempo exigido a 10^9 decriptações/s	Tempo exigido a 10^{13} decriptações/s
56	DES	$2^{56} \approx 7,2 \times 10^{16}$	2^{55} ns = 1,125 ano	1 hora
128	AES	$2^{128} \approx 3,4 \times 10^{38}$	2^{127} ns = $5,3 \times 10^{21}$ anos	$5,3 \times 10^{17}$ anos
168	Triple DES	$2^{168} \approx 3,7 \times 10^{50}$	2^{167} ns = $5,8 \times 10^{33}$ anos	$5,8 \times 10^{29}$ anos
192	AES	$2^{192} \approx 6,3 \times 10^{57}$	2^{191} ns = $9,8 \times 10^{40}$ anos	$9,8 \times 10^{36}$ anos
256	AES	$2^{256} \approx 1,2 \times 10^{77}$	2^{255} ns = $1,8 \times 10^{60}$ anos	$1,8 \times 10^{56}$ ano
26 caracteres (permutação)	Monoalfabético	$2! = 4 \times 10^{26}$	2×10^{26} ns = $6,3 \times 10^9$ anos	$6,3 \times 10^6$ anos

Número de Rodadas no DES

- Maior número de rodadas $\rightarrow$ mais difícil a criptoanálise, mesmo se a função F for relativamente fraca.
- Critério de projeto: número de rodadas suficiente para que criptoanálises conhecidas exijam mais esforço do que um ataque de força bruta.
- No DES:
 - 16 rodadas
 - Criptoanálise diferencial: $2^{55,1}$ operações
 - Força bruta: 2^{55} operações
 - Se houvesse 15 ou menos rodadas, criptoanálise diferencial exigiria menos esforço que a força bruta.
- Critério facilita comparar a força de diferentes algoritmos.
- Sem descobertas revolucionárias em criptoanálise, a força de um algoritmo que satisfaz este critério é julgada principalmente pelo tamanho da chave.

Projeto da Função F no DES

- A função F é o núcleo da cifra de bloco de Feistel e é responsável pela **confusão**.
- Critérios importantes para o projeto de F :
 - **Não linearidade**: quanto menos linear, mais difícil a criptoanálise.
 - **Efeito avalanche**: mudança em 1 bit da entrada altera muitos bits da saída.
 - **Strict Avalanche Criterion (SAC)** [WEBS86]:
 - Qualquer bit de saída muda com probabilidade $1/2$ quando qualquer bit de entrada é invertido.
 - Originalmente definido para S-boxes, mas aplicável à função F como um todo.
 - **Bit Independence Criterion (BIC)** [WEBS86]:
 - Bits de saída mudam independentemente quando qualquer bit de entrada é invertido.
- Aplicar SAC e BIC fortalece a eficácia da função de confusão e aumenta a segurança do algoritmo.

Algoritmo de Escalonamento de Chave

- Em cifras de bloco de Feistel, a chave principal é usada para gerar uma **subchave** para cada rodada.
- Objetivo do escalonamento de chave:
 - Maximizar a dificuldade de deduzir subchaves individuais.
 - Tornar mais difícil recuperar a chave principal a partir das subchaves.
- Não existe, até o momento, um princípio geral universalmente aceito para o projeto do algoritmo de escalonamento de chaves.
- O cuidado no escalonamento é crucial para garantir a segurança global do algoritmo.

Triple DES (3DES)

- Primeiro padronizado em 1985 pela ANSI para aplicações financeiras (X9.17).
- Incorporado ao DES como parte do padrão FIPS 46-3 em 1999.
- Utiliza **três chaves** e três execuções do algoritmo DES.
- Sequência de operação: **Encrypt-Decrypt-Encrypt (EDE)**:

$$C = E(K_3, D(K_2, E(K_1, P)))$$

- C = texto cifrado
- P = texto claro
- K_1, K_2, K_3 = chaves
- A abordagem EDE garante compatibilidade com DES original quando $K_1 = K_2 = K_3$.

Diretrizes do FIPS 46-3 para 3DES

- 3DES é o algoritmo de cifra simétrica aprovado pelo FIPS para uso corrente.
- O DES original (chave de 56 bits) é permitido apenas para sistemas legados.
- Novas aquisições de sistemas devem suportar 3DES.
- Organizações governamentais com sistemas DES legados são incentivadas a migrar para 3DES.
- Espera-se que 3DES e AES coexistam como algoritmos aprovados pelo FIPS, permitindo uma transição gradual para AES.

Exemplo do Efeito Avalanche no DES - Mudança na chave

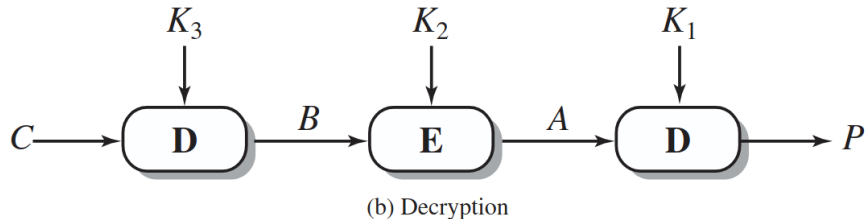
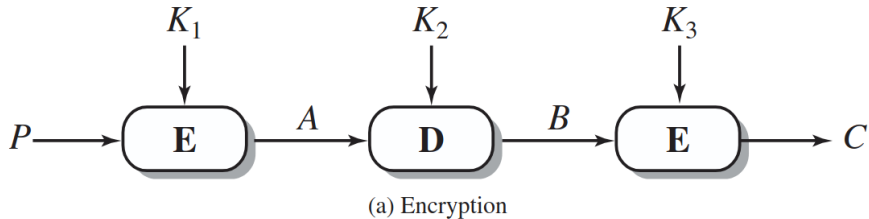


Figure 20.2 Triple DES

Compatibilidade do 3DES com DES

- Quando as três chaves são iguais ($K_1 = K_2 = K_3$), temos:

$$C = E(K_1, D(K_1, E(K_1, P))) = E(K, P)$$

- A operação do meio $D(K_1, E(K_1, P))$ se anula, resultando apenas na cifra DES original.
- Importância:
 - Permite que dados cifrados com DES possam ser decifrados por um sistema 3DES configurado em modo compatível.
 - Sistemas antigos que usam DES podem processar dados cifrados por 3DES no modo compatível.
- Essa retrocompatibilidade facilitou a adoção do 3DES sem necessidade de substituir todos os sistemas legados.

Chaves e Criptografia no 3DES

- O uso da deciptação na segunda etapa do 3DES não tem significado criptográfico; sua vantagem é permitir compatibilidade com o DES original:

$$C = E(K_1, D(K_1, E(K_1, P))) = E(K, P)$$

- Com três chaves distintas (K_1, K_2, K_3), o 3DES possui **168 bits efetivos de chave**.
- É possível usar apenas duas chaves ($K_1 = K_3$), resultando em uma chave efetiva de **112 bits**.
- O padrão FIPS 46-3 fornece diretrizes formais para a utilização do 3DES com estas configurações.

Chaves e Criptografia no 3DES

- O uso da deciptação na segunda etapa do 3DES não tem significado criptográfico; sua vantagem é permitir compatibilidade com o DES original:

$$C = E(K_1, D(K_1, E(K_1, P))) = E(K, P)$$

- Com três chaves distintas (K_1, K_2, K_3), o 3DES possui **168 bits efetivos de chave**.
- É possível usar apenas duas chaves ($K_1 = K_3$), resultando em uma chave efetiva de **112 bits**.
- O padrão FIPS 46-3 fornece diretrizes formais para a utilização do 3DES com estas configurações.

Advanced Encryption Standard (AES) e Rijndael

- Publicado pelo **NIST** em 2001 como padrão de cifra simétrica de bloco.
- Substitui o DES em diversas aplicações, oferecendo maior segurança e eficiência.
- AES é mais complexo que cifras de chave pública (ex.: RSA), sendo difícil de explicar de forma simplificada.
- Em 2000, o NIST selecionou a família de cifras de bloco **Rijndael** como vencedora do concurso AES.
- **Rijndael**: cifra de bloco escolhida como AES pelo NIST.
- **AES não usa Cifra de Feistel**
- Três variantes especificadas: AES-128; AES-192; AES-256

Table 3. Key-Block-Round Combinations

	Key length		Block size		Number of rounds
	Nk	(in bits)	Nb	(in bits)	Nr
AES-128	4	128	4	128	10
AES-192	6	192	4	128	12
AES-256	8	256	4	128	14

Diferenças entre AES-128, AES-192 e AES-256

- As três variantes diferem em três aspectos principais:
 1. **Comprimento da chave** (128, 192 ou 256 bits).
 2. **Número de rodadas (Nr)**, que determina o tamanho do key schedule.
 3. **Especificação da recursão em KEY EXPANSION()**.
- Para cada algoritmo:
 - **Nr**: número de rodadas.
 - **Nk**: número de palavras da chave.
 - **Nb**: número de palavras do estado; no padrão AES, **Nb = 4**.
- Somente essas configurações de Rijndael estão conformes com o padrão AES.
- [Veja a especificação FIPS 197 do AES](#)

AES também suporta vários modos

Exemplos de modos de operação:

- AES-ECB (Electronic Codebook)
- AES-CBC (Cipher Block Chaining)
- AES-CTR (Counter Mode)
- AES-GCM (Galois/Counter Mode)
- [Artigo: A Comparative Analysis of AES Common Modes of Operation](#)
- [Artigo: MODES OF OPERATION OF THE AES ALGORITHM](#)

Electronic Code Book (ECB) – Considerações

- O modo ECB divide a mensagem em blocos $(P_1, P_2, \dots, P_N)$ e cifra cada bloco separadamente com a mesma chave K .
- Se a mensagem não preencher o último bloco, ele é completado com *padding*.
- Vantagem: erros em um bloco não se propagam para os outros, permitindo decifrar blocos não corrompidos.
- Desvantagem: a criptografia é determinística; blocos de texto claro idênticos produzem blocos cifrados idênticos.
- Problemas adicionais:
 - Blocos idênticos ou mensagens com início igual são facilmente reconhecíveis.
 - A ordem dos blocos cifrados pode ser alterada sem que o receptor perceba.
- Conclusão: não recomendado para dados maiores que um bloco; alguns autores desaconselham completamente seu uso.

Modo ECB

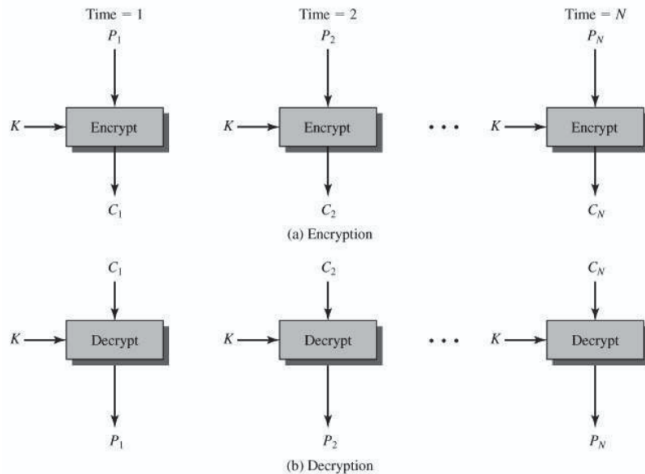


Figure 1: Scheme of the ECB mode of operation [2]

Cipher Block Chaining (CBC) – Considerações

- O modo CBC resolve o problema do ECB, reduzindo padrões repetidos no texto cifrado.
- Cada bloco de texto claro é XORado com o bloco cifrado anterior antes da encriptação.
- O primeiro bloco de texto claro é XORado com um *initialization vector* (IV).
- Benefício: mensagens longas com padrões repetidos podem ser tratadas de forma mais segura.
- Resultados de cifras distintas: mesmo texto claro cifrado múltiplas vezes gera diferentes textos cifrados devido ao IV.
- Desvantagem: requer mais tempo de processamento que o ECB devido ao encadeamento.
- Pode ser sincronizado para evitar propagação de erros causados por ruído no canal.
- O CBC não suporta paralelismo (**diferentemente do ECB**)

Modo CBC

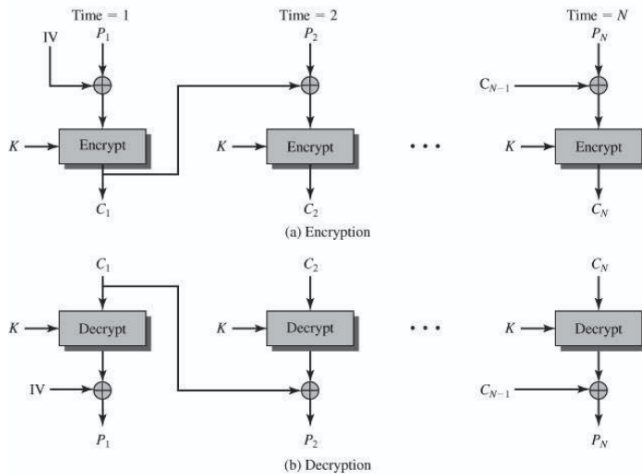
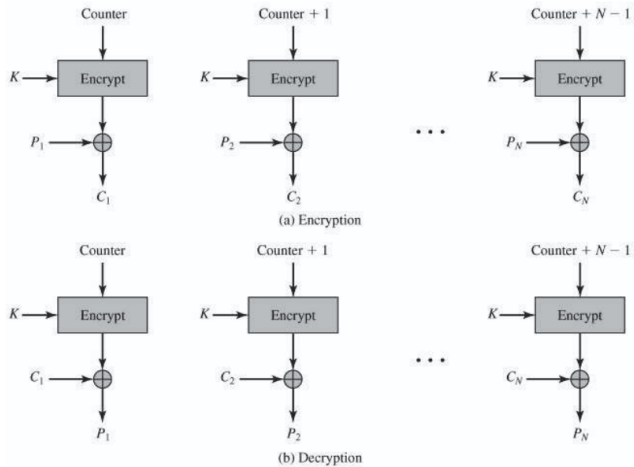


Figure 2: Scheme of the CBC mode of operation [2]

Counter Mode (CTR)

- Usa um contador como vetor de inicialização (IV), com tamanho igual ao do bloco.
- Cada bloco de texto claro é XORado com a saída da cifra aplicada ao contador.
- Não há necessidade de *padding* no último bloco.
- Os blocos são independentes, sem propagação de erro entre eles.
- Suporta paralelismo e pré-processamento, acelerando cifragem/decifragem.
- As operações de encriptação e decifração são idênticas.
- É fundamental não reutilizar o mesmo contador com a mesma chave, sob risco de quebra completa da confidencialidade.
- Normalmente, o contador é inicializado com um valor único (ex.: 96 bits aleatórios + 32 bits incrementais).
- A chave deve ser trocada após $2^{n/2}$ blocos (onde n é o tamanho do bloco).
- Considerado um dos modos mais seguros e eficientes para AES.

Modo CTR



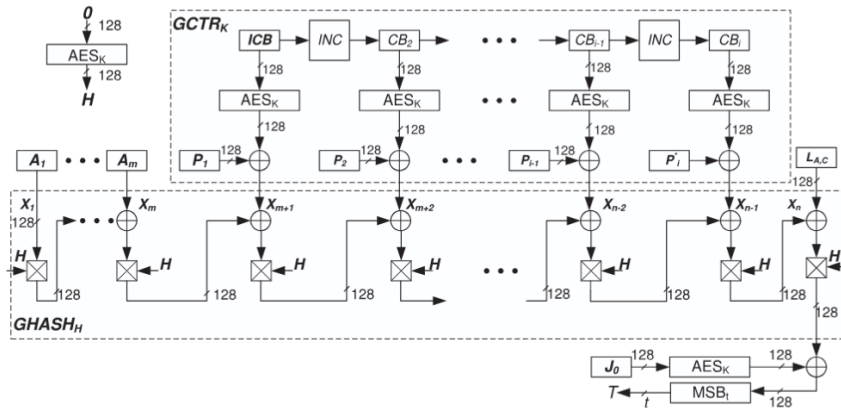
AES – Galois/Counter Mode (GCM)

- GCM combina duas funções:
 - **Autenticação**: cálculo de um *tag* de integridade.
 - **Confidencialidade**: criptografia no modo *Counter*.
- O processo de confidencialidade é baseado no modo CTR (*Counter Mode*).
- A autenticação é realizada pela função **GHASH**, que usa multiplicações em $GF(2^{128})$.
 - $GF(2^{128})$ é o campo de Galois com 128 bits: cada bloco de 128 bits é tratado como um polinômio de grau ≤ 127 com coeficientes 0 ou 1; a soma é XOR e a multiplicação é feita módulo

$$p(x) = x^{128} + x^7 + x^2 + x + 1.$$

- O *hash subkey* H é obtido aplicando AES sobre o bloco nulo.
- O *tag* de autenticação T é gerado a partir dos dados confidenciais e dos dados adicionais autenticados (AAD).
- Na decifração autenticada, o *tag* é verificado para garantir integridade e autenticidade.
- [Artigo: Modo GCM](#)

Modo GCM



AES-GCM: Campo de Galois e GHASH

- O GCM usa o campo finito $GF(2^{128})$ para autenticação.
- Cada bloco de 128 bits é tratado como um polinômio de grau até 127:

$$b_0 + b_1x + b_2x^2 + \cdots + b_{127}x^{127}, \quad b_i \in \{0, 1\}$$

- A chave de hash H é gerada cifrando um bloco de zeros com AES.
- O GHASH combina cada bloco de dados ou AAD com H assim:

$$X_i = (X_{i-1} \oplus B_i) \cdot H \mod p(x)$$

- O polinômio irreducível usado é fixo pelo padrão NIST:

$$p(x) = x^{128} + x^7 + x^2 + x + 1$$

Fluxo de autenticação no AES-GCM (GHASH)

O fluxo de autenticação no AES-GCM (GHASH) funciona assim:

1. Pega o acumulador do bloco anterior X (ou zero no primeiro bloco).
2. Faz XOR com o bloco atual da mensagem P .
3. Multiplica o resultado pelo H , que é a chave de hash obtida cifrando um bloco de zeros com AES.
4. Reduz módulo o polinômio

$$p(x) = x^{128} + x^7 + x^2 + x + 1$$

para manter 128 bits.

Ou seja, cada passo é:

$$X_i = ((X_{i-1} \oplus P_i) \cdot H) \bmod p(x)$$

- O XOR encadeia os blocos e mistura os dados.
- O módulo garante que o resultado continue com 128 bits, pronto para o próximo bloco ou para gerar a Tag final.

Aula 10 - Funções Hash

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
June 2, 2026

Funções de Hash

- Uma função de hash aceita uma mensagem de tamanho variável M como entrada.
- Produz um valor de tamanho fixo $h = H(M)$.
- O valor h é chamado de **hash** ou **digest**.

Propriedades Desejáveis de Hash

- A saída deve parecer **aleatória** e estar **uniformemente distribuída**.
- Uma pequena mudança em M altera, com alta probabilidade, muitos bits de h .
- Principal objetivo: **integridade de dados**.
- Exemplo: se qualquer bit de M for alterado, o hash $H(M)$ também mudará.

Função de Hash Criptográfica

- Tipo especial de função de hash usada em aplicações de segurança.
- Deve ser **computacionalmente inviável** de quebrar com eficiência maior que força bruta.
- Usada para verificar se os dados foram alterados.

Propriedades de uma Função de Hash Criptográfica

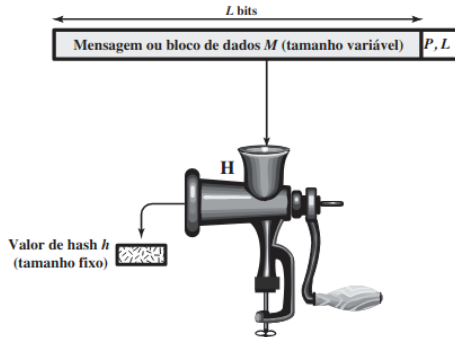
- **Mão única (one-way):** Dado um hash h , é inviável encontrar uma mensagem M tal que $H(M) = h$.
- **Livre de colisão (collision-free):** É inviável encontrar duas mensagens M_1 e M_2 tais que $H(M_1) = H(M_2)$.

Preenchimento em Funções de Hash

- Funções de hash processam mensagens em blocos de tamanho fixo.
- Quando a mensagem não é múltiplo do tamanho do bloco, adiciona-se **preenchimento** (padding).
 - Mensagem preenchida até se tornar um múltiplo de um tamanho fixo (ex: 1024 bits).
- O preenchimento inclui o **tamanho original da mensagem** em bits.
 - **Objetivo:** dificultar que um atacante crie uma mensagem alternativa com o mesmo hash.
- Garante que cada mensagem de tamanho diferente resulte em um hash único e seguro.

Preenchimento em Funções de Hash

Figura 11.1 Função de hash criptográfica; $h = H(M)$.



P, L = preenchimento mais campo de tamanho

Uso do Hash:

- Ela é usada em diversas aplicações de segurança e protocolos da Internet.
- Talvez o hash seja o algoritmo criptográfico mais versátil.

Uso onde a Hash é empregada:

- Autenticação de mensagem
- Assinaturas digitais
- Arquivo de senha de mão única
- Detecção de intrusão e detecção de vírus
- Função pseudoaleatória (PRF)
- Gerador de número pseudoaleatório (PRNG)

Autenticação de Mensagem

- Autenticação de mensagem é um mecanismo usado para verificar a integridade de uma mensagem
- Garante que os dados recebidos estão exatamente como foram enviados, sem modificação, inserção, exclusão ou repetição.
- Em muitos casos, também é exigido que a identidade declarada do emissor seja validada.
- Em muitas aplicações, o valor gerado pelo Hash é chamado de **resumo de mensagem** (ou *digest*, em inglês).

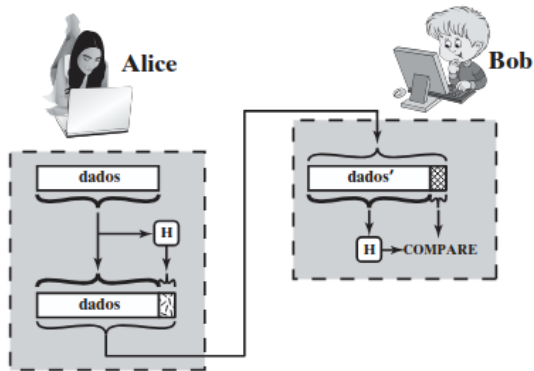
Autenticação de mensagem

A essência do uso de uma função de hash para autenticação de mensagem é a seguinte:

1. O emissor calcula um valor de hash como função dos bits da mensagem.
2. O emissor transmite a mensagem junto com o valor de hash.
3. O receptor recalcula o valor de hash sobre a mensagem recebida.
4. O receptor compara o valor calculado com o valor recebido.

Se houver divergência, o receptor sabe que a mensagem (ou o valor de hash) foi alterada.

Autenticação básica



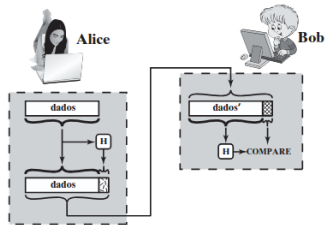
(a) Uso da função de hash para verificar integridade de dados

Pergunta

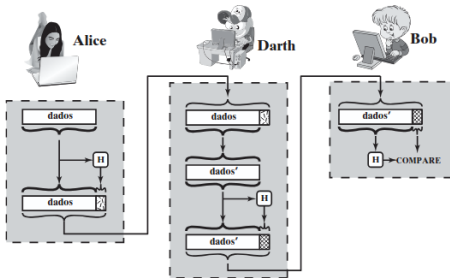
Qual o problema neste cenário?

Autenticação básica - Problema

Figura 11.2 Ataque contra função de hash.



(a) Uso da função de hash para verificar integridade de dados



(b) Ataque *man-in-the-middle*

Proteção do valor de hash

A função de hash precisa ser **transmitida de forma segura**.

- Se um adversário **alterar ou substituir** a mensagem, não deve ser viável alterar também o valor de hash para enganar o receptor.
- Exemplo de ataque:
 1. Alice transmite dados com o hash
 2. Darth intercepta, altera a mensagem e calcula um novo hash
 3. Bob recebe sem perceber a modificação.
- Para impedir esse ataque, **o valor de hash gerado por Alice precisa ser protegido**.

Pergunta

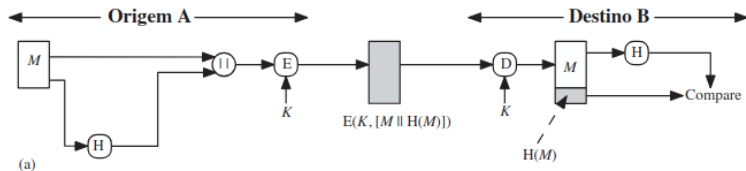
Como podemos proteger o Hash nesse cenário?

Métodos de proteção da Hash

- Método A: Autenticação com hash + cifragem simétrica
- Método B: Cifrando o hash com cifragem simétrica
- Método C: Mensagem com Hash e Valor Secreto
- Método D: Mensagem com Hash mais Valor Secreto com confidencialidade

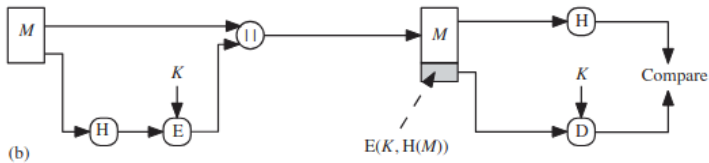
Método A: Autenticação com hash + cifragem simétrica

- Mensagem mais o código de hash concatenado **são encriptados** usando a encriptação simétrica.
- Como somente A e B compartilham a chave secreta, a mensagem deverá ter vindo de A e sem alteração.
- O código de hash oferece a estrutura ou redundância exigida para conseguir a autenticação.
- Como a encriptação é aplicada à mensagem inteira mais o código de hash, a confidencialidade também é fornecida.



Método B: Cifrando o hash com cifragem simétrica

- Somente o código de hash é encriptado, usando a encriptação simétrica.
- Isso reduz o peso do processamento para as **aplicações que não exigem confidencialidade**.



Pergunta

Qual o motivo de passar a mensagem em texto plano?

Vantagens do Uso de Hash mas sem a cifragem da mensagem

- Quando a **confidencialidade não é exigida**:
 - usar apenas hash (método com valor secreto) requer menos cálculos que cifrar a mensagem inteira.
 - O software de encriptação é relativamente lento, especialmente com fluxo constante de mensagens.
 - Custos de hardware de encriptação podem ser altos; chips de baixo custo existem, mas cada nó precisa ter capacidade.

Exemplo desse cenário

Baixando a .iso do Linux Mint e verificando com GPG

O que o GPG faz com esses dois arquivos?

Comando:

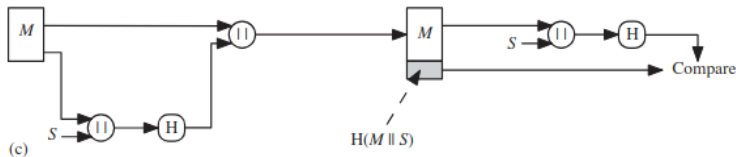
```
gpg --verify sha256sum.txt.gpg sha256sum.txt
```

Etapas realizadas pelo GPG:

1. Lê a assinatura digital contida em sha256sum.txt.gpg.
2. Calcula o hash real do conteúdo atual de sha256sum.txt.
3. Compara o hash calculado com o hash assinado.
 - Se forem **iguais**: a assinatura é válida (Good signature).
 - Se forem **diferentes**: a assinatura falha (BAD signature).

Método C: Mensagem com Hash e Valor Secreto

- Usar apenas uma função de hash, sem cifragem, para autenticação de mensagem.
- Ambas as partes compartilham um valor secreto comum: S .
- Funcionamento:
 1. Emissor (A) calcula o hash sobre a concatenação da mensagem e do segredo: $H(M \parallel S)$.
 2. O valor de hash resultante é anexado à mensagem e enviado a B.
 3. Receptor (B), possuindo S , recalcula o hash para verificar a integridade e autenticidade.
- Como S não é enviado, um adversário não consegue modificar a mensagem nem gerar mensagens falsas.



Método C é a base do HMAC - Hash Based Message Authentication Code

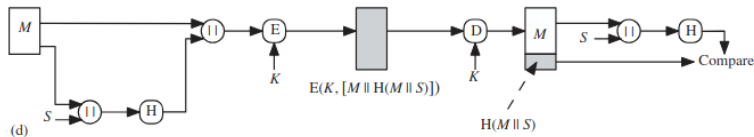
- A autenticação de mensagem normalmente é alcançada usando um **MAC** (Message Authentication Code), também chamado de função de hash chaveada.
- MACs são usados entre duas partes que compartilham uma **chave secreta** para autenticar informações trocadas.
- A função MAC recebe como entrada a chave secreta e um bloco de dados, produzindo um valor de hash (**MAC**) associado à mensagem.

Código de Autenticação de Mensagem (MAC)

- Para verificar integridade, aplica-se novamente a função MAC sobre a mensagem e compara-se com o MAC recebido.
- Um invasor que altere a mensagem não poderá gerar o MAC correto sem conhecer a chave secreta.
- A verificação também garante autenticidade: apenas a parte que conhece a chave secreta pode ter gerado o MAC.

Método D: Mensagem com Hash mais Valor Secreto com confidencialidade

- A **confidencialidade** pode ser acrescentada à abordagem do método anterior encriptando a mensagem inteira mais o código de hash

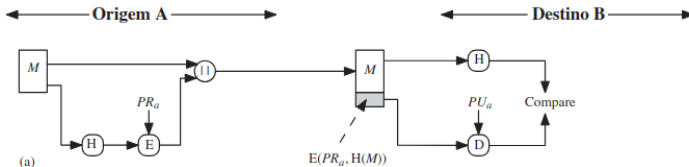


Cenário

Esse cenário mostra exatamente um canal criptografado com a autenticação da mensagem, igual na VPN!

Assinaturas Digitais

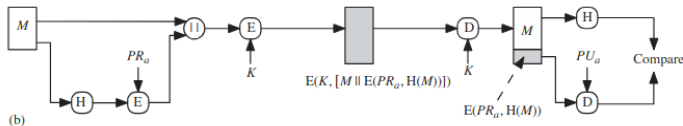
- Usa **Chave pública e privada**
- Assinatura digital é uma aplicação importante, similar à autenticação de mensagem.
- O valor de hash da mensagem é encriptado com a **chave privada** do usuário.
- Qualquer pessoa que conheça a **chave pública** do usuário pode verificar a integridade da mensagem associada à assinatura.
- Um invasor que tente alterar a mensagem precisaria conhecer a chave privada do usuário.



Esse caso é exatamente o caso do hash da iso do Linux Mint!

Assinaturas Digitais com Confidencialidade

- Se, além da assinatura digital, o que se procura é confidencialidade, então a mensagem mais o código hash encriptado com a chave privada pode ser encriptado usando uma chave secreta simétrica. Essa é uma técnica comum.



Arquivos de Senha de Mão Única

- Funções de hash são usadas para criar arquivos de senha de **mão única**.
 - No linux usa-se o `/etc/shadow`
- Em vez de armazenar a senha real, o sistema operacional armazena o **hash da senha**.
- Assim, mesmo que um hacker acesse o arquivo, a senha real não pode ser recuperada.
- Processo de autenticação:
 1. O usuário fornece a senha
 2. O sistema compara o **hash informado** com o hash armazenado.
- Este método é amplamente usado na maioria dos sistemas operacionais.

Detecção de Intrusão e Vírus com Hash

- Funções de hash podem ser usadas para **detecção de intrusão** e **detecção de vírus**.
- Armazene $H(F)$ para cada arquivo em um sistema e guarde os valores de hash de forma segura.
- Posteriormente, verifique se um arquivo foi modificado recalculando $H(F)$.
- Um intruso precisaria alterar F sem alterar $H(F)$ para evitar detecção, o que é computacionalmente inviável.

Trabalho interessante:

- [Documentação Labrador](#)
- [Código Labrador](#)

Além disso, Hashes são usadas em função pseudoaleatória (PRF) ou gerador de número pseudoaleatório (PRNG)

Funções de Hash Simples

- Para entender considerações de segurança, apresentamos funções de hash simples e **não seguras**.
- Todas as funções de hash operam sobre blocos de n bits da entrada (mensagem, arquivo etc.).
- A entrada é processada bloco a bloco em um padrão iterativo para produzir um hash de n bits.
- Um exemplo simples: o **XOR bit a bit** de cada bloco.

$$C_i = b_{i1} \oplus b_{i2} \oplus \dots \oplus b_{im}$$

onde

C_i = i -ésimo bit do código de hash, $1 \leq i \leq n$

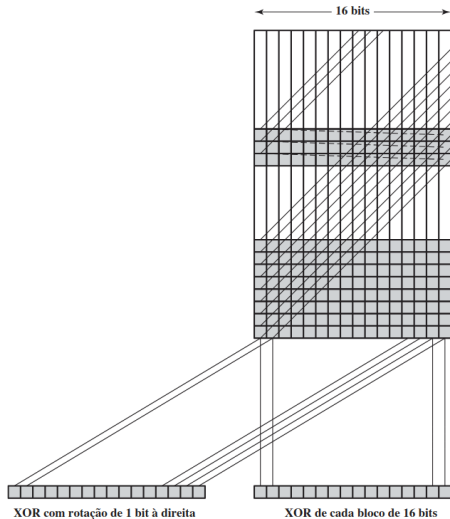
m = número de blocos de n bits na entrada

b_{ij} = i -ésimo bit no j -ésimo bloco

$\oplus$ = operação XOR

Esquema de Hash simples com XOR

Figura 11.5 Duas funções de hash simples.



Limitações do XOR em Funções de Hash

XOR simples não é suficiente:

- Problema: blocos de texto cifrado podem ser **reordenados** sem alterar o valor do hash.
- Isso permite que um atacante modifique a mensagem **sem que seja detectada mudanças na integridade** da informação.
- Conclusão: para garantir a integridade, precisamos de funções de hash criptográficas **resistentes a colisões**.

Pré-imagem e Colisões em Funções de Hash

- Para um valor de hash $h = H(x)$, x é chamado de **pré-imagem** de h .
 - Isso significa que x é um bloco de dados cuja função hash produz h .
- Funções hash são **mapas muitos-para-um**: para qualquer valor h , podem existir várias pré-imagens.
- **Colisão**: ocorre se existirem $x \neq y$ tal que $H(x) = H(y)$.
- Colisões são indesejáveis quando funções de hash são usadas para **integridade de dados**.

Pré-imagens e Potenciais Colisões

- Suponha uma função de hash H com **saída** de n bits e **entrada** de b bits ($b > n$).
- Total de mensagens de entrada possíveis: 2^b .
- Total de valores de hash possíveis: 2^n .
- Em média, cada valor de hash corresponde a 2^{b-n} pré-imagens.
- Se H distribui uniformemente os valores de hash, cada hash terá aproximadamente 2^{b-n} pré-imagens.
- Para entradas de tamanho variável, a variação de pré-imagens por valor de hash aumenta.
- Apesar disso, os riscos de segurança não são tão graves; é necessário **definir requisitos** precisos de segurança para funções de hash criptográficas.

Requisitos de Funções de Hash

Tabela 11.1 Requisitos para função de hash criptográfica H .

Requisito	Descrição
Tamanho de entrada variável	H pode ser aplicado em um bloco de dados de qualquer tamanho.
Tamanho da saída fixo	H produz uma saída de tamanho fixo.
Eficiência	$H(x)$ é relativamente fácil de calcular para qualquer valor de x informado, através de implementações tanto em hardware quanto em software.
Resistência à pré-imagem (propriedade de mão única)	Para qualquer valor de hash h informado, é computacionalmente impossível encontrar y , de modo que $H(y) = h$.
Resistência à segunda pré-imagem (resistência à colisão fraca)	Para qualquer bloco x informado, é computacionalmente impossível encontrar $y \neq x$ com $H(y) = H(x)$.
Resistência à colisão forte	É computacionalmente impossível encontrar qualquer par (x, y) , de modo que $H(x) = H(y)$.
Pseudoaleatoriedade	A saída de H atende os testes padrão de pseudoaleatoriedade.

As primeiras três propriedades são requisitos **básicos** de uma função de hash.

Resistência à Pré-imagem (Propriedade de Mão Única)

- A resistência à pré-imagem significa que:
 1. É fácil gerar o código de hash a partir da mensagem.
 2. Porém, é praticamente impossível gerar a mensagem a partir do código de hash.
- Se a função de hash não tiver esta propriedade, um invasor pode:
 - Observar a mensagem M e o hash $h = H(S||M)$.
 - Inverter a função de hash para obter $S||M = H^{-1}(h)$.
 - Recuperar o valor secreto S facilmente.
- Portanto, a resistência à pré-imagem é crucial para proteger valores secretos.
- Geralmente esse requisito é essencial quando a autenticação envolve um valor secreto que não é transmitido

Resistência à Segunda Pré-imagem

- Garante que é **impossível** encontrar uma mensagem alternativa com o mesmo valor de hash de uma mensagem específica.
- Se a função de hash não possuir essa propriedade, um invasor poderia:
 - Observar ou interceptar uma mensagem com seu hash encriptado.
 - Criar uma nova mensagem diferente com o mesmo hash.
- Portanto, esta propriedade protege contra **ataques de falsificação**.
- Ou seja, é importante considerar este aspecto para prevenção contra falsificação quando se usa um hash encriptado.

Funções Hash Fraca e Forte

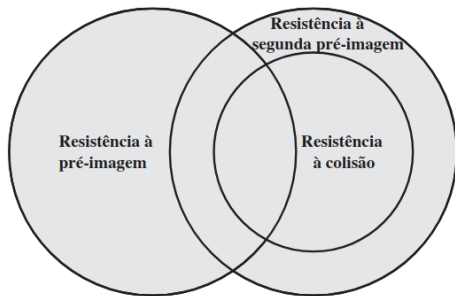
- Se uma função de hash **somente** satisfaz as primeiras cinco propriedades é chamada de **função hash fraca**.
- Se a função também satisfaz a sexta propriedade, **resistência à colisão**, é chamada de **função hash forte**.
- Exemplo de ataque sem resistência à colisão:
 - Bob cria duas mensagens diferentes com o mesmo hash (m_1 e m_2).
 - Alice assina a mensagem m_1 .
 - Bob usa o hash da mensagem m_1 para reivindicar que a mensagem m_2 foi assinada.
- Portanto, a resistência à colisão é crucial para evitar falsificação de assinaturas.
- **Motivo:** Função hash forte protege contra ataques onde terceiros tentam gerar mensagens diferentes com o mesmo hash.

Pseudoaleatoriedade em Funções de Hash Criptográficas

- A pseudoaleatoriedade **não é tradicionalmente citada como requisito formal**, mas é implicitamente importante.
- Funções de hash criptográficas são frequentemente usadas para:
 - Derivação de chaves.
 - Geração de números pseudoaleatórios (PRNG/PRF).
- Nas aplicações de integridade de mensagens, as propriedades de resistência dependem de a saída parecer aleatória.
- **Ou seja:** é apropriado considerar que uma função de hash produz uma saída pseudoaleatória.

Relação entre propriedades de Funções de Hash

Figura 11.6 Relação entre as propriedades das funções de hash.



- Uma função que é resistente à colisão **também** é resistente à segunda pré-imagem, **mas o inverso não é necessariamente verdadeiro**.
- Uma função pode ser resistente à colisão, mas não ser resistente à pré-imagem, e vice-versa. Uma função pode ser resistente à pré-imagem, mas não ser resistente à segunda pré-imagem e vice-versa.

Propriedades de resistência Funções de Hash e seu uso

Tabela 11.2 Propriedades de resistência necessárias para várias aplicações de integridade de dados.

	Resistência à pré-imagem	Resistência à segunda pré-imagem	Resistência à colisão
Hash + assinatura digital	sim	sim	sim*
Deteção de intrusão e detecção de vírus		sim	
Hash + encriptação simétrica			
Arquivo de senha de mão única	sim		
MAC	sim	sim	sim*

*Resistência necessária se o invasor é capaz de elaborar um determinado ataque de mensagem

Ataques de Força Bruta em Funções de Hash

Ataques de força bruta:

- **Não dependem** do algoritmo de hash: dependem **apenas do tamanho do valor de hash** em bits.
- Consiste em tentar todas as combinações possíveis até encontrar uma pré-imagem ou colisão.
- O esforço necessário cresce exponencialmente com o tamanho do hash:
 - Para um hash de n bits: há 2^n possíveis valores.
 - Exemplo: para um hash de 128 bits, seriam necessárias 2^{128} tentativas em média para encontrar uma colisão por força bruta.

Lembrando:

- Força bruta é diferente da criptoanálise
- Criptoanálise explora vulnerabilidades específicas do algoritmo

Ataques de Pré-imagem e Segunda Pré-imagem

Cenário: O atacante que conhece uma saída da Hash deseja:

- O adversário busca um valor y tal que $H(y) = h$, onde h é um hash conhecido.
- Esse ataque explora a **dificuldade de inverter** a função de hash (resistência à pré-imagem)
- Método de **força bruta**:
 - Testar valores aleatórios de y até encontrar uma correspondência.
 - Para um hash de m bits, o esforço médio necessário é 2^{m-1} tentativas.

Ataques Resistentes à Colisão

Cenário: O atacante busca descobrir **pares de saída** da Hash

- O adversário busca duas mensagens x e y tal que $H(x) = H(y)$.
- Esse ataque exige **menos esforço** do que um ataque de pré-imagem ou segunda pré-imagem.
- Para um hash de m bits, o esforço médio para encontrar uma colisão é aproximadamente $2^{m/2}$ tentativas.

Lembra do **paradoxo do aniversário**?

- Dado um grupo de 23 pessoas escolhidas aleatoriamente, a chance de que duas pessoas terão a mesma data de aniversário é de mais de 50
- Em hashes a probabilidade de colisão aumenta rapidamente com o número de tentativas.

Ataque de Colisão Explorado via Paradoxo do Aniversário

Estratégia para explorar o paradoxo do dia do aniversário:

- Preparação na origem: mensagem legítima x é criada
- Oponente gera $2^{m/2}$ variações x' de x com mesmo significado e armazena os hashes.
- Oponente prepara uma mensagem fraudulenta y para a qual deseja a assinatura.
- Pequenas variações y' de y são geradas; o oponente calcula $H(y')$ e verifica correspondência com algum $H(x')$.
- Quando há correspondência, a variação válida de A é usada para assinatura, que é então aplicada à variação fraudulenta y' .

Ambas produzem a mesma assinatura.

Exemplo de Ataque de Colisão com Hash de 64 bits

- Com um hash de 64 bits, o esforço necessário é da ordem de 2^{32} .
- Criação de variações que mantêm o mesmo significado não é difícil.
- Exemplos de variações:
 - Inserção de pares de caracteres “espaço-espaço-retrocesso” entre palavras.
 - Substituição de “espaço-retrocesso-espaço” em posições selecionadas.
 - Reescrita da mensagem mantendo o significado original.

Resumo do esforço exigido

Resumo: Para um código de hash de tamanho m , o nível de esforço exigido, conforme vimos, é proporcional ao seguinte:

Resistência à pré-imagem	2^m
Resistência à segunda pré-imagem	2^m
Resistência à colisão	$2^{m/2}$

Resistência a Colisões em Hashes

- A resistência à colisão é desejável em códigos de hash seguros.
- Para um hash de m bits, a força contra ataques por força bruta é aproximadamente $2^{m/2}$.
- Van Oorschot e Wiener [VANO94] projetaram uma máquina de US\$10 milhões para MD5 (128 bits):
 - Capaz de encontrar uma colisão em 24 dias.
 - Mostra que 128 bits é inadequado para segurança moderna.
- Hashes de 160 bits (como SHA-1) aumentam a resistência:
 - Mesma máquina levaria mais de 4.000 anos para encontrar uma colisão.
 - Contudo, com a evolução tecnológica: 160 bits começa a não ser mais seguro.

Saindo da força bruta e entrando na criptoanálise:

- Assim como em algoritmos de criptografia, ataques criptoanalíticos em **hashes** e **MACs** buscam explorar propriedades do algoritmo para superar a simples busca exaustiva.
- A resistência de um hash ou MAC à criptoanálise é medida comparando-se seu esforço com o esforço de um ataque por força bruta.
- Um hash ou MAC ideal exigirá **esforço criptoanalítico maior ou igual ao esforço por força bruta**.

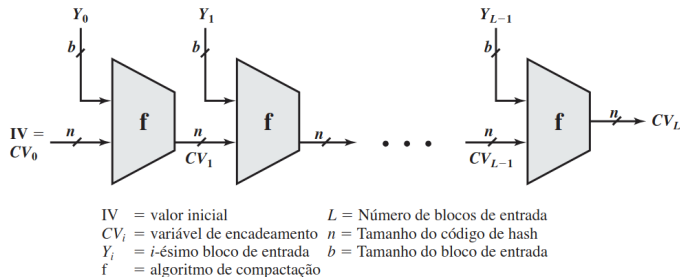
Estrutura Iterativa de Funções de Hash

Estrutura geral de uma típica função de hash segura:

- Estrutura iterativa de hash foi proposta por Merkle [MERK79, MERK89]
- É usada na maioria das funções de hash atuais, incluindo SHA.
- Funcionamento geral:
 - A mensagem de entrada é dividida em L blocos de tamanho fixo b bits.
 - O bloco final é preenchido para completar b bits (Se necessário)
 - Inclui também o tamanho total da mensagem.
 - Incluir o tamanho dificulta ataques, pois o oponente deve encontrar colisões com mensagens do mesmo ou de tamanhos diferentes que levem ao mesmo hash.

Estrutura geral de hash seguro

Figura 11.8 Estrutura geral do código de hash seguro.



O algoritmo usa repetidamente uma **função de compactação f** :

- Duas entradas: (i) a **variável de encadeamento** (n bits da etapa anterior) e (ii) bloco atual (b bits).
- Produz uma saída de n bits. Normalmente, $b > n$;
- Variável de encadeamento inicial é **definida pelo algoritmo** e o valor final dela é o **hash resultante**.

A função de hash pode ser resumida da seguinte forma:

$$CV_0 = IV = \text{valor inicial de } n \text{ bits}$$

$$CV_i = f(CV_{i-1}, Y_{i-1}) \quad 1 \leq i \leq L$$

$$H(M) = CV_L$$

onde a entrada da função de hash é uma mensagem M consistindo nos blocos $Y_0, Y_1, \dots, Y_{L-1}$

Motivação da estrutura iterativa (Merkle [MERK89], Damgård [DAMG89]):

- Se a função de compactação f for à prova de colisão, a função de hash iterativa resultante também será.
- Permite criar hashes seguros para mensagens de qualquer tamanho.
- O design seguro de uma função de hash se reduz ao design de uma função de compactação segura para blocos de tamanho fixo.

Criptanálise de funções de hash:

- Foca na estrutura interna de f .
- Ataques procuram produzir colisões eficientes para uma única execução de f , considerando o valor fixo do IV.
- Normalmente, f consiste em várias rodadas, e o ataque analisa padrões de mudança de bits entre rodadas.

- **Importante:** Para qualquer função de hash, **colisões sempre existem:**
 - Mensagens têm tamanho $\geq 2b$ (devido ao campo de tamanho)
 - Hashes têm tamanho fixo n , com $b > n$
- O objetivo de uma função de hash segura não é eliminar colisões (isso é impossível), mas torná-las **computacionalmente inviáveis de encontrar**.
- **Importante:** A segurança é definida pelo **esforço necessário para descobrir uma colisão, não pela sua inexistência**.

Secure Hash Algorithm (SHA)

- O **SHA** é a função de hash mais utilizada nos últimos anos.
- Em 2005, era praticamente o último algoritmo de hash padronizado restante após vulnerabilidades em outros algoritmos.
- Desenvolvido pelo **NIST** e publicado como padrão federal (**FIPS 180**) em 1993.
- Primeira versão (**SHA-0**) apresentou vulnerabilidades criptoanalíticas.
- Revisão lançada em 1995 (**FIPS 180-1**), conhecida como **SHA-1**.
- Baseado na função de hash **MD4**, seguindo de perto seu projeto.

- **SHA-1** produz um hash de 160 bits.
- Em 2002, o **NIST** revisou o padrão (**FIPS 180-2**), definindo três novas versões:
 - SHA-256 (256 bits)
 - SHA-384 (384 bits)
 - SHA-512 (512 bits)
- Coletivamente, estas versões são conhecidas como **SHA-2**.
- Mantêm a mesma estrutura básica do SHA-1, usando aritmética modular e operações binárias lógicas.
- Em 2008, o **FIPS PUB 180-3** adicionou uma versão de 224 bits (SHA-224).
- SHA-1 e SHA-2 também são especificados no **RFC 6234**, que inclui implementação em C.

Descontinuação do SHA-1

- Em 2005, o **NIST** anunciou a intenção de retirar a aprovação do SHA-1 e adotar o SHA-2 por volta de 2010.
- Uma equipe de pesquisa demonstrou um ataque que poderia gerar duas mensagens diferentes com o mesmo hash SHA-1 usando 2^{69} operações.
- Este número é significativamente menor que as 2^{80} operações anteriormente estimadas para encontrar uma colisão.
- O resultado acelerou a necessidade de transição para SHA-2.
- Referência: Wang et al. [WANG05].

Tabela 11.3 Comparação de parâmetros do SHA.

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Tamanho do resumo da mensagem	160	224	256	384	512
Tamanho da mensagem	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Tamanho do bloco	512	512	512	1024	1024
Tamanho da word	32	32	32	64	64
Número de etapas	80	64	64	80	80

Nota: todos os tamanhos são medidos em bits.

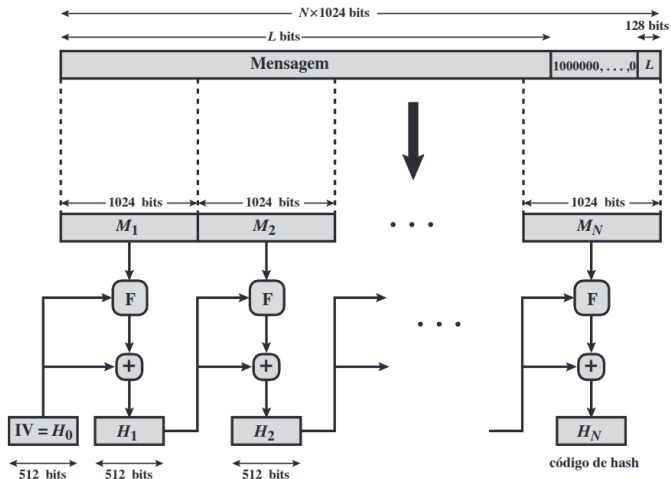
- **Entrada:** mensagem com um tamanho máximo de menos de 2^{128} bits
- **Processamento:** A entrada é processada em blocos de 1024 bits.
- **Saída:** Resumo da mensagem de 512 bits.

O processamento consiste nas seguintes etapas:

1. Anexar bits de preenchimento.
2. Anexar tamanho.
3. Inicializar buffer de hash.
4. Processar mensagem em blocos de 1024 bits

Processamento SHA-512

Figura 11.9 Geração de resumo da mensagem usando SHA-512.



Etapa 1 - Preenchimento:

- A mensagem é preenchida para que o tamanho seja congruente a 896 módulo 1024.
- O preenchimento é sempre aplicado, mesmo se a mensagem já tiver o tamanho desejado.
- Número de bits de preenchimento: entre 1 e 1024.
- Estrutura do preenchimento: um bit 1 seguido pelos bits 0 necessários.

Etapa 2 - Anexar tamanho:

- Um bloco de 128 bits é anexado à mensagem.
- O bloco contém o tamanho da mensagem original (antes do preenchimento) como um inteiro de 128 bits sem sinal.
- Ordem dos bytes: byte mais significativo primeiro.

Obs:

- Após as duas primeiras etapas, a mensagem resultante tem comprimento múltiplo de 1024 bits.
- Mensagem expandida representada como blocos de 1024 bits: $M_1, M_2, \dots, M_N$.
- Tamanho total da mensagem expandida: $N \times 1024$ bits.

SHA-512: Etapa 3 - Inicialização do Buffer de Hash

- Um buffer de 512 bits mantém resultados intermediários e finais da função de hash.
- O buffer pode ser representado por 8 registradores de 64 bits, inicializados com seguintes valores hexadecimais ($64 \times 8 = 512$):

a =	6A09E667F3BCC908	e =	510E527FADE682D1
b =	BB67AE8584CAA73B	f =	9B05688C2B3E6C1F
c =	3C6EF372FE94F82B	g =	1F83D9ABFB41BD6B
d =	A54FF53A5F1D36F1	h =	5BE0CD19137E2179

- Esses valores são armazenados no formato big-endian, que é o byte mais significativo de uma word na posição de byte de endereço baixo (mais à esquerda).
- Essas words foram obtidas apanhando-se os primeiros sessenta e quatro bits das partes fracionárias das raízes quadradas dos oito primeiros números primos.

SHA-512: Etapa 4 - Processar mensagem em blocos de 1024 bits

O núcleo do algoritmo SHA-512 processa a mensagem em blocos de **1024 bits** (128 words). Esse processamento consiste em um módulo de **80 rodadas**

- Esse módulo é o que está rotulado como F na figura.

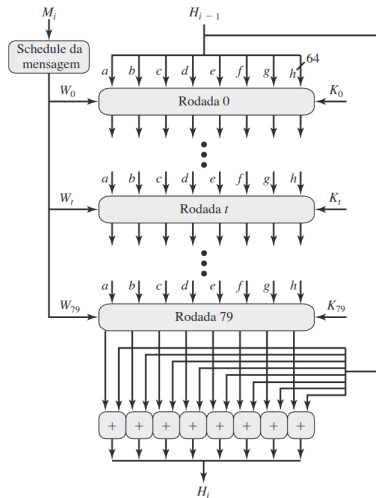
Cada rodada recebe como entrada o **buffer de 512 bits** (a, b, c, d, e, f, g, h) e o atualiza.

- Na primeira rodada, o buffer contém o hash intermediário H_{i-1} .
- Cada rodada t utiliza:
 - Um valor de 64 bits W_t , derivado do bloco atual M_i (schedule da mensagem).
 - Uma constante aditiva, K_t , onde $0 \leq t \leq 79$ indica uma das 80 rodadas

As constantes K_t são os primeiros 64 bits das partes fracionárias das raízes cúbicas dos 80 primeiros números primos, oferecendo padrões pseudoaleatórios que reduzem regularidades na entrada.

Processamento do SHA-512 em um único bloco

Figura 11.10 Processamento SHA-512 de um único bloco de 1024 bits.



Constantes do SHA-512

Tabela 11.4 Constantes do SHA-512.

428a2f98d728ae22	7137449123ef65cd	b5c0fbcfec4d3b2f	e9b5dba58189dbbc
3956c25bf348b538	59f111f1b605d019	923f82a4af194f9b	ab1c5ed5da6d8118
d807aa98a3030242	12835b0145706fbe	243185be4ee4b28c	550c7dc3d5ffb4e2
72be5d74f27b896f	80deb1fe3b1696b1	9bdc06a725c71235	c19bf174cf692694
e49b69c19ef14ad2	efbe4786384f25e3	0fc19dc68b8cd5b5	240ca1cc77ac9c65
2de92c6f592b0275	4a7484aa6ea6e483	5cb0a9dcbbd41fbd4	76f988da831153b5
983e5152ee66dfab	a831c66d2db43210	b00327c898fb213f	bf597fc7beef0ee4
c6e00bf33da88fc2	d5a79147930aa725	06ca6351e003826f	142929670a0e6e70
27b70a8546d22ffc	2e1b21385c26c926	4d2c6dfc5ac42aed	53380d139d95b3df
650a73548baf63de	766a0abb3c77b2a8	81c2c92e47edaee6	92722c851482353b
a2bfe8a14cf10364	a81a664bbc423001	c24b8b70d0f89791	c76c51a30654be30
d192e819d6ef5218	d69906245565a910	f40e35855771202a	106aa07032bbd1b8
19a4c116b8d2d0c8	1e376c085141ab53	2748774cdf8eeb99	34b0bcb5e19b48a8
391c0cb3c5c95a63	4ed8aa4ae3418acb	5b9cca4f7763e373	682e6ff3d6b2b8a3
748f82ee5defb2fc	78a5636f43172f60	84c87814a1f0ab72	8cc702081a6439ec
90befffa23631e28	a4506cebde82bde9	bef9a3f7b2c67915	c67178f2e372532b
ca273eceeaa26619c	d186b8c721c0c207	eada7dd6cde0eb1e	f57d4f7fee6ed178
06f067aa72176fba	0a637dc5a2c898a6	113f9804bef90dae	1b710b35131c471b
28db77f523047d84	32caab7b40c72493	3c9ebe0a15c9bebc	431d67c49c100d4c
4cc5d4becb3e42b6	597f299cfc657e2a	5fcb6fab3ad6faec	6c44198c4a475817

SHA-512: Etapa 5 - Saída

Depois que todos os N blocos de 1024 bits tiverem sido processados, a saída do estágio N é o resumo de mensagem de 512 bits.

Podemos resumir o comportamento do SHA-512 da seguinte forma:

$$H_0 = IV$$

$$H_i = \text{SUM}_{64}(H_{i-1}, \text{abcdefgh}_i)$$

$$MD = H_N$$

onde

IV = valor inicial do buffer abcdefgh , definido na etapa 3

abcdefgh_i = a saída da última rodada de processamento do i -ésimo bloco de mensagem

N = o número de blocos na mensagem (incluindo campos de preenchimento e tamanho)

SUM_{64} = adição módulo 2^{64} realizada separadamente em cada word do par de entradas

MD = valor final do resumo da mensagem

Aula 11 - Criptografia Assimétrica

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
June 2, 2026

A criptografia de chave pública introduz uma mudança fundamental:

- Baseada em **funções matemáticas**, e não apenas em substituição e permutação.
- É **assimétrica**: utiliza duas chaves separadas (pública e privada), ao contrário da simétrica que usa uma única chave.
- Essa assimetria impacta diretamente a **confidencialidade**, a **distribuição de chaves** e a **autenticação**.
- Grande parte da teoria dos criptossistemas de chave pública é baseada na teoria dos números.

Concepções equivocadas comuns:

- Não é intrinsecamente mais segura que a criptografia simétrica — a segurança depende do **tamanho da chave** e do **custo computacional** para quebrar a cifra.
- Não substitui a criptografia simétrica — mas apresenta toda uma gama de aplicações:
 - **Gerenciamento de chaves**
 - **Assinaturas digitais**

Chaves Assimétricas: Duas chaves relacionadas, uma **pública** e uma **privada**, usadas para operações complementares:

- Encriptação e deciptação
- Geração e verificação de assinatura

Certificado de Chave Pública: Documento emitido e assinado digitalmente pela chave privada de uma **Autoridade de Certificação (CA)**:

- Vincula o **nome de um assinante** a uma **chave pública**.
- Garante que o assinante identificado tem **controle exclusivo** da chave privada correspondente.

Algoritmo Criptográfico Assimétrico: Algoritmo criptográfico que usa duas chaves relacionadas:

- Uma chave **pública** e uma chave **privada**.
- É computacionalmente inviável derivar a chave privada a partir da pública.

Infraestrutura de Chave Pública (PKI): Conjunto de políticas, processos e plataformas para administração de certificados e pares de chaves:

- Emissão, manutenção e revogação de certificados.
- Envolve servidores, software e estações de trabalho.
- Suporte para autenticação, confidencialidade e integridade.

Como Surgiu a Criptografia Assimétrica

O conceito de criptografia de chave pública evoluiu de uma tentativa de atacar **dois dos problemas mais difíceis associados à encriptação simétrica**:

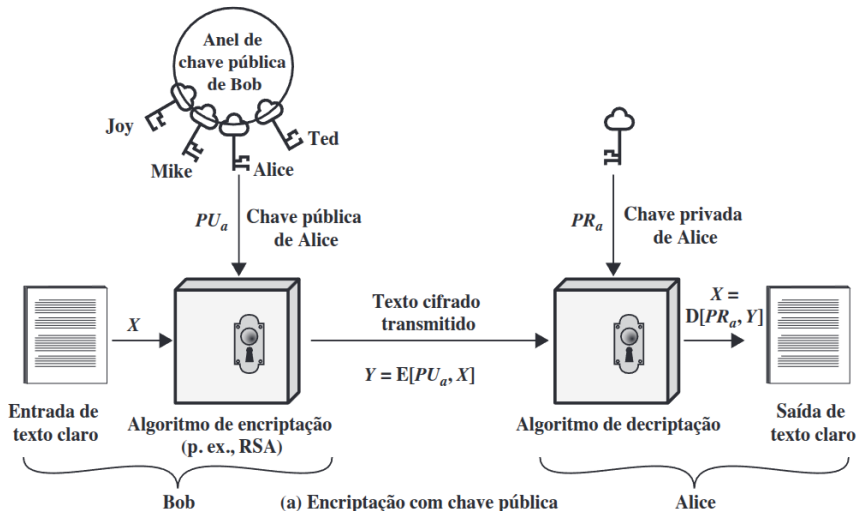
- **A distribuição de chaves** sob a encriptação simétrica:
 - Requer que dois comunicantes já compartilhem uma chave, que de alguma forma foi distribuída a eles; **OU**
 - Requer o uso de um centro de distribuição de chaves - Diffie [DIFF88]: “afinal, qual seria a vantagem de desenvolver criptossistemas impenetráveis, se seus usuários fossem forçados a compartilhar suas chaves com um CDC que poderia ser comprometido por roubo ou suborno?”
- **Assinaturas digitais** - Se o uso da criptografia tivesse que se tornar comum, não apenas nas situações militares, mas para fins comerciais e particulares, então as mensagens e documentos eletrônicos precisariam do equivalente das assinaturas usadas nos documentos em papel.

Os algoritmos assimétricos contam com uma chave para encriptação e uma chave diferente, porém relacionada, para a decifração. Eles têm a seguinte característica importante:

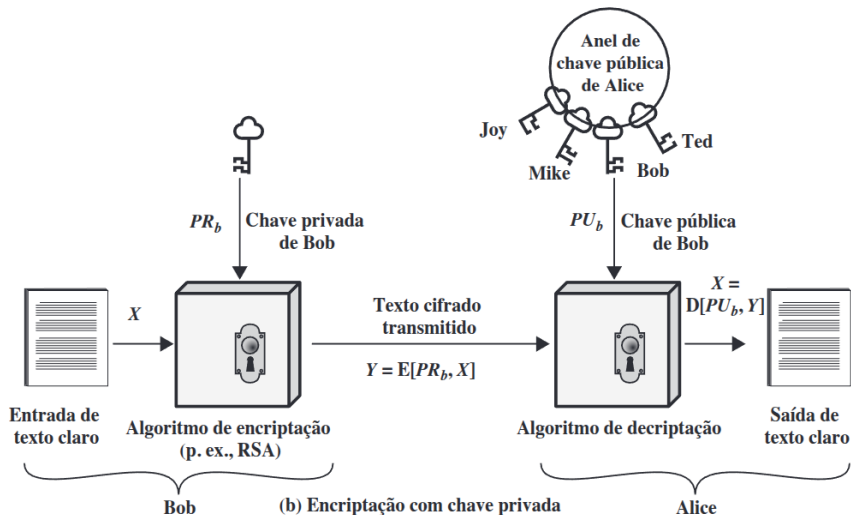
- É computacionalmente inviável determinar a chave de decifração dado apenas o conhecimento do algoritmo de criptografia e da chave de encriptação.
- Além disso, alguns algoritmos, como RSA, também exibem esta característica: Qualquer uma das duas chaves relacionadas pode ser usada para encriptação, com a outra para a decifração.

Encriptação com chave pública

Figura 9.1 Criptografia de chave pública.



Encriptação com chave privada



Etapas Essenciais da Criptografia de Chave Pública

1. Cada usuário gera um **par de chaves** (pública e privada) para encriptação e deciptação.
2. A **chave pública** é disponibilizada em um repositório ou arquivo acessível; a **chave privada** permanece secreta.
3. Para enviar uma mensagem confidencial a Alice, Bob usa a **chave pública de Alice** para encriptar a mensagem.
4. Ao receber, Alice utiliza sua **chave privada** para deciptar. Apenas ela consegue recuperar o texto original.

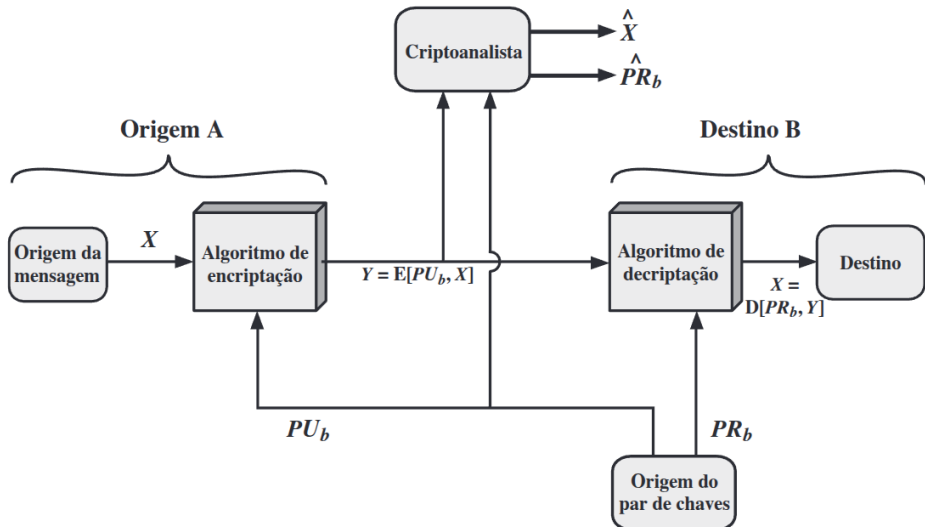
Esquema de Encriptação de Chave Pública

Um esquema de encriptação de chave pública possui cinco elementos principais:

1. **Texto claro:** mensagem ou dados legíveis fornecidos como entrada.
2. **Algoritmo de encriptação:** aplica transformações no texto claro.
3. **Chaves pública e privada:** par de chaves; se uma encripta, a outra decripta.
4. **Texto cifrado:** mensagem embaralhada gerada como saída, dependente da chave.
5. **Algoritmo de decriptação:** reverte o processo, recuperando o texto claro original.

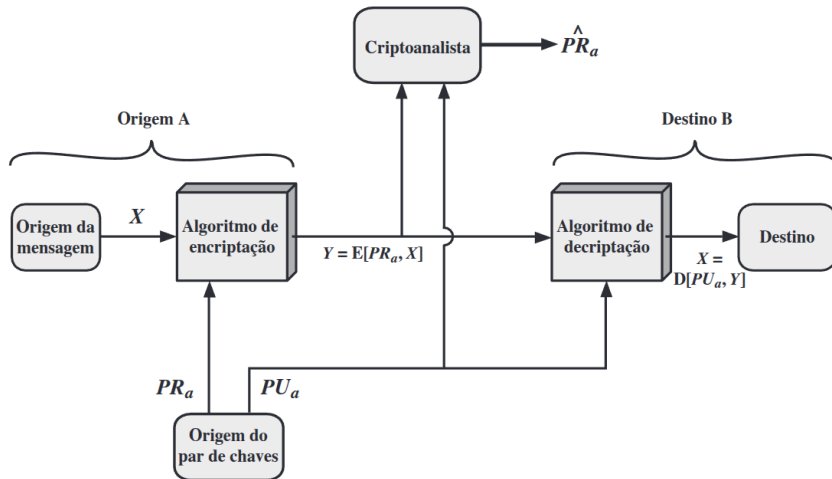
Criptossistema de chave pública - Objetivo Sigilo

Figura 9.2 Criptossistema de chave pública: sigilo.



Criptossistema de chave pública - Objetivo autenticação

Figura 9.3 Criptossistema de chave pública: autenticação.



Autenticação e Confidencialidade com Chave Pública

Importante:

- Apenas a encriptação com a chave privada do emissor garante **autenticação**, mas não a confidencialidade.
- Para prover **ambos**, aplica-se uma dupla encriptação:

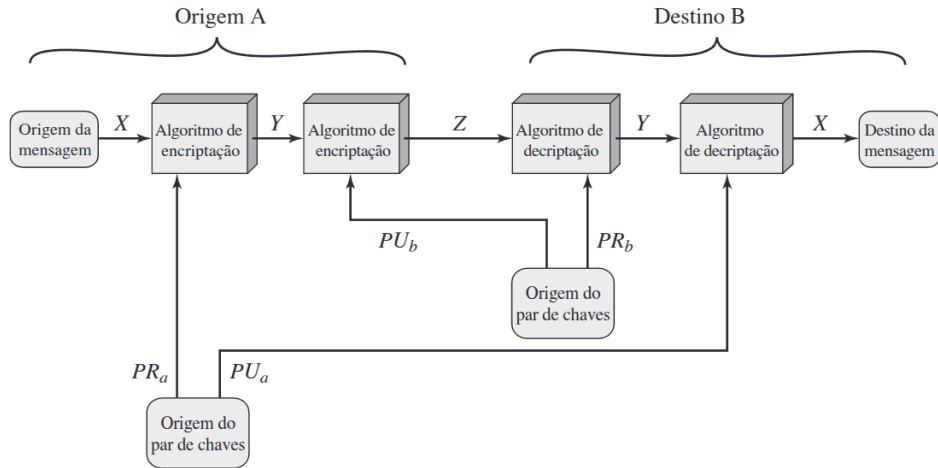
$$Z = E(PU_b, E(PR_a, X))$$

$$X = D(PU_a, D(PR_b, Z))$$

- O emissor primeiro encripta com sua **chave privada** (assinatura digital).
- Em seguida, encripta novamente com a **chave pública do receptor**.
- O receptor, ao decifrar, recupera tanto a **confidencialidade** quanto a **autenticidade**.
- Desvantagem: exige 4 operações de criptografia assimétrica por mensagem.

Criptossistema de chave pública - Objetivo autenticação e sigilo

Figura 9.4 Criptossistema de chave pública: autenticação e sigilo.



Aplicações de criptografia de chave pública

Quadro 9.3 Aplicações para criptosistemas de chave pública.

ALGORITMO	ENCRIPÇÃO/ DECRIPÇÃO	ASSINATURA DIGITAL	TROCA DE CHAVE
RSA	Sim	Sim	Sim
Curva elíptica	Sim	Sim	Sim
Diffie-Hellman	Não	Não	Sim
DSS	Não	Sim	Não

Obs: DSS é a sigla para Digital Signature Standard, um padrão de assinatura digital que, em conjunto com o DSA (Digital Signature Algorithm) e o SHA (Secure Hash Algorithm), permite a criação de assinaturas digitais e verificação de integridade de dados, utilizando criptografia de chave pública de forma a garantir a autenticidade e a não-repudição de uma transação

Requisitos para Criptografia de Chave Pública

1. Geração de par de chaves (PU_b, PR_b) deve ser **computacionalmente fácil**.
2. Encriptação: dado PU_b e M , calcular $C = E(PU_b, M)$ deve ser **fácil**.
3. Decriptação: dado PR_b e C , calcular $M = D(PR_b, C)$ deve ser **fácil**.
4. **Inviável** obter PR_b a partir de PU_b .
5. **Inviável** recuperar M conhecendo apenas PU_b e C .
6. (Opcional) Ordem das chaves pode ser invertida:
$$M = D[PU_b, E(PR_b, M)] = D[PR_b, E(PU_b, M)].$$

Observação: Apenas poucos algoritmos atendem a todos os requisitos, como RSA, ECC, Diffie-Hellman e DSS.

Função de Mão Única com Alçapão

- Função de mão única: mapeia um domínio X em um intervalo Y de forma que:

$Y = f(X)$ é fácil de calcular, mas $X = f^{-1}(Y)$ é inviável de obter.

- "Com alçapão": existe uma informação secreta que permite inverter a função de forma eficiente.
- Essencial para criptografia de chave pública: permite encriptação segura sem compartilhar a chave privada.

Complexidade de Funções Criptográficas

- **Fácil:** problema resolvível em tempo polinomial $O(n^a)$ em função do tamanho da entrada n . Onde a é uma constante fixa.
- Algoritmos polinomiais pertencem à classe P.
- **Inviável:** problema cujo esforço cresce mais rápido que o tempo polinomial, por exemplo, $O(2^n)$.
- Observação: medir complexidade pelo pior caso ou caso médio não é suficiente para criptografia.
- Criptografia exige que a função seja inviável de reverter para **praticamente todas as entradas**.

Função de Mão Única com Alçapão

- Uma função que é fácil de calcular em uma direção e inviável na outra, a menos que informação adicional (alçapão) seja conhecida.
- Formalmente, uma família de funções reversíveis f_k :
 - $Y = f_k(X)$ é fácil, se k e X forem conhecidos.
 - $X = f_k^{-1}(Y)$ é fácil, se k e Y forem conhecidos.
 - $X = f_k^{-1}(Y)$ é inviável, se apenas Y for conhecido, sem k .
- A construção de esquemas de **chave pública** depende de **encontrar funções de mão única com alçapão adequadas**.

- O artigo de Diffie e Hellman (1976) desafiou a comunidade a criar algoritmos de chave pública. [Referência](#)
- Muitos algoritmos iniciais se mostraram falhos.
- Em 1977, Ron Rivest, Adi Shamir e Len Adleman (MIT) desenvolveram o esquema RSA. [Referência](#)
- Publicado em 1978, o RSA se tornou a técnica mais aceita e amplamente implementada para criptografia de chave pública.

Funcionamento do RSA

- RSA é uma cifra de bloco: o texto claro e o texto cifrado são inteiros entre 0 e $n - 1$.
- Tamanho típico: $n \approx 1024$ bits (cerca de 309 dígitos decimais).
- Cada bloco de texto claro M deve satisfazer $0 \leq M < n$.
- Encriptação e deciptação:

$$C = M^e \bmod n,$$

$$M = C^d \bmod n = (M^e)^d \bmod n$$

- Chave pública: $PU = \{e, n\}$
Chave privada: $PR = \{d, n\}$
- Requisitos:
 1. $M^{ed} \bmod n = M$ para todo $M < n$
 2. Calcular $M^e \bmod n$ e $C^d \bmod n$ é eficiente
 3. Conhecendo apenas e e n , é inviável determinar d

Requisito de Inversão no RSA

- Para garantir que $M^{ed} \bmod n = M$, é necessário que:
 - os expoentes e e d sejam **inversos multiplicativos módulo** $\varphi(n)$,
 - onde $\varphi(n)$ é a **função totiente de Euler**.
- Para $n = pq$, com p e q primos:

$$\varphi(n) = (p - 1)(q - 1)$$

- Relação fundamental:

$$ed \equiv 1 \pmod{\varphi(n)} \Rightarrow d \equiv e^{-1} \pmod{\varphi(n)}$$

- O inverso multiplicativo **só existe** se e for **coprime com** $\varphi(n)$:

$$\text{mdc}(e, \varphi(n)) = 1$$

- O valor de d pode ser obtido com o **algoritmo de Euclides estendido**, que encontra o inverso modular de e em relação a $\varphi(n)$.

Inverso Multiplicativo no RSA

No RSA, a chave privada d é escolhida como o inverso multiplicativo de e módulo $\varphi(n)$:

$$d \equiv e^{-1} \pmod{\varphi(n)}$$

- Isso significa que d satisfaz:

$$d \cdot e \equiv 1 \pmod{\varphi(n)}$$

- Exemplo: $e = 7$, $\varphi(n) = 160$

$$23 \cdot 7 = 161 \equiv 1 \pmod{160}$$

- Portanto, $d = 23$ é o inverso multiplicativo de $e = 7$ módulo 160.

Entendendo como funciona: O que queremos garantir

No RSA:

- Mensagem M
- Chave pública (n, e)
- Chave privada (n, d)

Criptografia:

$$C \equiv M^e \pmod{n}$$

Descriptografia:

$$M' \equiv C^d \pmod{n} = (M^e)^d \pmod{n}$$

Objetivo:

$$M' \equiv M \pmod{n}$$

Entendendo como funciona: Por que e e d devem ser inversos módulo $\varphi(n)$

- $n = pq$, $\varphi(n) = (p - 1)(q - 1)$
- Teorema de Euler: $M^{\varphi(n)} \equiv 1 \pmod{n}$ se $\text{mdc}(M, n) = 1$
- Escolhemos d tal que:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

- Ou seja, existe k tal que $ed = 1 + k\varphi(n)$

Entendendo como funciona: Aplicando na descryptografia

$$M' = C^d = (M^e)^d = M^{ed} = M^{1+k\varphi(n)} = M \cdot (M^{\varphi(n)})^k$$

Pelo Teorema de Euler:

$$(M^{\varphi(n)})^k \equiv 1^k \equiv 1 \pmod{n}$$

Portanto:

$$M' \equiv M \pmod{n}$$

Conclusão: funciona porque $e \cdot d \equiv 1 \pmod{\varphi(n)}$

Segurança do RSA

- **1. Estrutura das chaves:**

- Chave pública: $PU = \{e, n\}$
- Chave privada: $PR = \{d, n\}$
- Onde $n = p \cdot q$, e p e q são primos grandes e secretos

- **2. Se p e q forem descobertos:**

- É possível calcular o totiente de Euler:

$$\varphi(n) = (p - 1)(q - 1)$$

- E então obter o inverso modular de e :

$$d \equiv e^{-1} \pmod{\varphi(n)}$$

- Logo, o atacante consegue reconstruir a chave privada d

- **3. Conclusão:**

- A segurança do RSA depende da dificuldade de **fatorar** n .
- Se alguém conseguir obter p e q , o sistema é completamente quebrado.
- Portanto, manter p e q secretos é **fundamental**.

Escolha do valor e no RSA

- **Geralmente, escolhemos** $e = 65537 = 2^{16} + 1$
- É considerado o “**ponto ideal**” entre segurança e desempenho.

Propriedades:

- **Ímpar**: requisito para ser coprimo com $\varphi(n)$ na maioria dos casos.
- **Pequeno o bastante para ser rápido**:
 - Tem apenas 17 bits.
 - Exige pouquíssimas multiplicações na exponenciação modular.
- **Grande o suficiente para ser seguro**:
 - Evita ataques que exploram expoentes muito pequenos (como $e = 3$ ou $e = 17$).
- **Forma binária simples**:

$$e = 10000000000000001_2$$

o que torna a operação de exponenciação modular bem eficiente.

Esquema RSA

Ingredientes:

- Dois números primos secretos: p e q .
- Calcula-se $n = pq$ (público).
- Escolhe-se e tal que $\text{mdc}(\varphi(n), e) = 1$ e $1 < e < \varphi(n)$ (público).
- Calcula-se $d \equiv e^{-1} \pmod{\varphi(n)}$ (privado).

Chaves:

- Chave privada: $\{d, n\}$
- Chave pública: $\{e, n\}$

Processo de encriptação e decríptação:

- Usuário B quer enviar mensagem M para A:

$$C = M^e \pmod{n} \quad (\text{encriptação})$$

- Usuário A recebe C e decrípta:

$$M = C^d \pmod{n}$$

Tabela resumo dos parâmetros do RSA

Figura 9.5 O algoritmo RSA.

Geração de chave por Alice	
Selecione p, q	p e q são primos, $p \neq q$
Calcule $n = p \times q$	
Calcule $\phi(n) = (p - 1)(q - 1)$	
Selecione o inteiro e	$\text{mdc}(\phi(n), e) = 1; 1 < e < \phi(n)$
Calcule d	$d \equiv e^{-1} \pmod{\phi(n)}$
Chave pública	$PU = \{e, n\}$
Chave privada	$PR = \{d, n\}$
Encriptação por Bob com chave pública de Alice	
Texto claro:	$M < n$
Texto cifrado:	$C = M^e \pmod{n}$
Deciptação por Alice com a chave privada de Alice	
Texto cifrado:	C
Texto claro:	$M = C^d \pmod{n}$

Exemplo Numérico do RSA

Geração de chaves:

- $p = 17, q = 11$
- $n = pq = 187$
- $\varphi(n) = (p - 1)(q - 1) = 160$
- Escolhe $e = 7, \text{mdc}(e, 160) = 1$
- Calcula $d \equiv e^{-1} \pmod{160} \Rightarrow d = 23$

Explicação

- Exemplo: $e = 7, \varphi(n) = 160$

$$23 \cdot 7 = 161 \equiv 1 \pmod{160}$$

- Portanto, $d = 23$ é o inverso multiplicativo de $e = 7$ módulo 160.

Chaves:

- Pública: $PU = \{7, 187\}$
- Privada: $PR = \{23, 187\}$

Exemplo Numérico do RSA

Encriptação de $M = 88$:

$$C = M^e \mod n = 88^7 \mod 187$$

$$88^7 \mod 187 = (88 \times 88^2 \times 88^4) \mod 187 = (88 \times 77 \times 132) \mod 187 = 11$$

Decriptação:

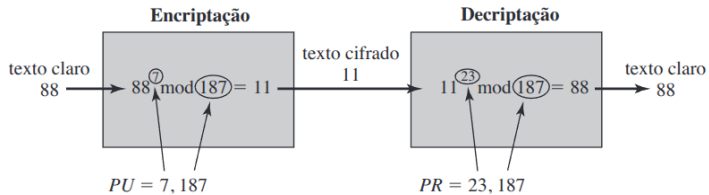
$$M = C^d \mod n = 11^{23} \mod 187$$

$$11^{23} \mod 187 = 88$$

Resultado: A mensagem original $M = 88$ é recuperada com sucesso.

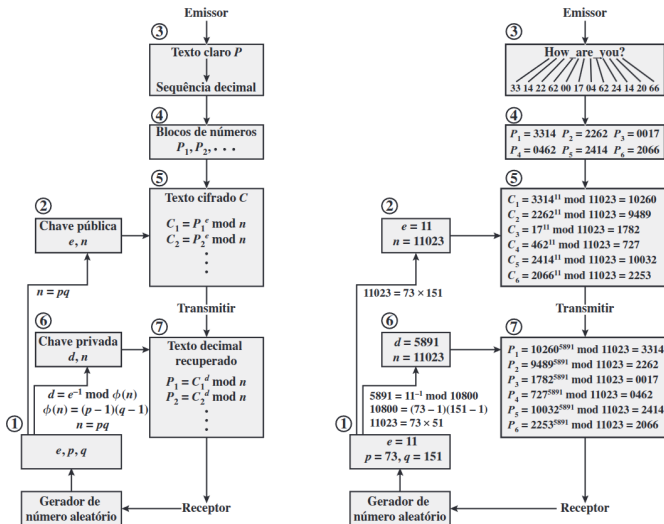
Exemplo Simples numérico - RSA

Figura 9.6 Exemplo de algoritmo RSA.



Processamento de múltiplos blocos usando RSA

Figura 9.7 Processamento de múltiplos blocos com RSA.



Usando e verificando os parâmetros RSA na prática

- **Gerar a chave ssh:**

```
ssh-keygen -t rsa -f ./teste
```

- **Verificar fingerprint da chave:**

```
ssh-keygen -lf ./teste
```

- **Tentear abrir** (não vai funcionar):

```
openssl rsa -in ./teste -text -noout
```

- **Mudar o formato:**

```
ssh-keygen -p -m PEM -f ./teste
```

- **Verificar fingerprint anterior:**

```
ssh-keygen -lf ./teste
```

- **Tentear abrir** (vai funcionar):

```
openssl rsa -in ./teste -text -noout
```

- **Mudar formato da chave pública para modo PEM:**

```
ssh-keygen -e -m PEM -f ./teste.pub > ./teste_pub.pem
```

- **Abrir a chave publica:**

```
openssl rsa -pubin -in teste_pub.pem -text -noout
```

O que é cada parâmetro de uma chave RSA

Nome	Símbolo	Função
modulus	$n = p \cdot q$	Base da aritmética modular — o número grande
publicExponent	e	Expoente público, normalmente 65537.
privateExponent	d	Expoente privado usado para assinar/decifrar.
prime1	p	Primeiro primo secreto.
prime2	q	Segundo primo secreto.
exponent1 / exponent2	$d \bmod (p - 1)$ e $d \bmod (q - 1)$	Para otimização - Teorema Chinês do Resto.
coefficient	$q^{-1} \bmod p$	Outro parâmetro auxiliar do Teo. Chin. Resto.

O Problema da Fatoração no RSA

A segurança do RSA depende da dificuldade de certos problemas matemáticos. Existem três possíveis formas de ataque:

1. **Fatorar n :** encontrar os primos p e q tais que $n = p \times q$. Isso permite calcular $\varphi(n) = (p - 1)(q - 1)$ e, em seguida, $d \equiv e^{-1} \pmod{\varphi(n)}$.
2. **Determinar $\varphi(n)$ diretamente:** sem fatorar n , mas que leva igualmente à descoberta de d .
3. **Descobrir d diretamente:** a partir de e e n , sem precisar de $\varphi(n)$. Até hoje, parece tão difícil quanto fatorar n .

Conclusão:

- A criptoanálise do RSA está fortemente ligada ao problema da **fatoração de inteiros**.
- O desempenho dos melhores algoritmos de fatoração é usado como **benchmark de segurança**.

Dificuldade de Fatoração em Diferentes Tamanhos de Chave RSA

- A dificuldade de fatorar cresce rapidamente com o tamanho do módulo:
 - Fatorar um módulo RSA de **1024 bits** é cerca de **mil vezes mais difícil** do que um de 768 bits.
 - Um módulo de **768 bits** é milhares de vezes mais difícil que um de 512 bits.
- A primeira fatoração bem-sucedida de um módulo de **512 bits** ocorreu há cerca de uma década.
- É plausível que um módulo de **1024 bits** possa ser fatorado dentro da próxima década por grupos acadêmicos.

Recomendação prática:

- Evitar o uso de chaves RSA de 1024 bits nos próximos **3 a 4 anos**.
- Preferir módulos de pelo menos **2048 bits** para segurança a médio prazo.

Troca de Chaves Diffie-Hellman

- Primeiro algoritmo de chave pública, proposto por Diffie e Hellman em 1976.
- Objetivo: permitir que dois usuários troquem **variáveis** para criar uma chave secreta com segurança.
- Essa chave secreta pode ser usada posteriormente para criptografar mensagens.
- O algoritmo em si não criptografa mensagens, apenas realiza a **troca de valores secretos**.
- Baseado em propriedades da aritmética modular e do problema do logaritmo discreto.

Logaritmo Discreto e Diffie-Hellman

- A segurança do algoritmo Diffie-Hellman depende da dificuldade de calcular **logaritmos discretos**.
- Definição: dado um número primo p e uma raiz primitiva a , temos:

$$a^1 \bmod p, a^2 \bmod p, \dots, a^{p-1} \bmod p$$

que geram todos os inteiros de 1 até $p - 1$.

- Para qualquer inteiro b , existe um único expoente i tal que:

$$b \equiv a^i \pmod{p}, \quad 0 \leq i \leq p - 1$$

- Esse i é o **logaritmo discreto** de b na base a , módulo p :

$$i = \log_a(b) \pmod{p}$$

- Problema: calcular i é computacionalmente inviável para valores grandes de p .

Algoritmo de Troca de Chaves Diffie-Hellman

- Dois parâmetros públicos:

$$q \text{ (número primo)}, \quad \alpha \text{ (raiz primitiva de } q)$$

- Cada usuário escolhe um valor secreto:

$$A : X_A < q, \quad B : X_B < q$$

- Cada um calcula sua chave pública:

$$Y_A = \alpha^{X_A} \bmod q, \quad Y_B = \alpha^{X_B} \bmod q$$

- As chaves públicas Y_A e Y_B são trocadas.
- Cálculo da chave secreta compartilhada:

$$A : K = (Y_B)^{X_A} \bmod q$$

$$B : K = (Y_A)^{X_B} \bmod q$$

- Como $(Y_B)^{X_A} \equiv (Y_A)^{X_B} \pmod{q}$, ambos obtêm a mesma chave secreta K .

Calculo da chave K - Diffie-Hellman

A chave K é calculada por ambas as partes da seguinte forma:

$$\begin{aligned} K &= (Y_B)^{X_A} \bmod q \\ &= (\alpha^{X_B} \bmod q)^{X_A} \bmod q \\ &= (\alpha^{X_B})^{X_A} \bmod q \\ &= \alpha^{X_B X_A} \bmod q \\ &= (\alpha^{X_A})^{X_B} \bmod q \\ &= (\alpha^{X_A} \bmod q)^{X_B} \bmod q \\ &= (Y_A)^{X_B} \bmod q \end{aligned}$$

pelas regras da aritmética modular

Troca de Chaves Diffie-Hellman

O resultado é que os dois lados trocaram um valor secreto. Esse valor secreto é normalmente usado como chave secreta simétrica compartilhada.

Agora, considere um intruso que pode observar a troca de chaves e queira determinar a chave secreta K . Como X_A e X_B são privados, o intruso só tem os seguintes elementos para trabalhar: q , α , Y_A e Y_B .

Assim, o intruso é forçado a calcular um **logaritmo discreto** para estabelecer a chave. Por exemplo, para definir a chave privada do usuário B, o intruso precisaria calcular:

$$X_B = \log_{\alpha}(Y_B) \pmod{q}$$

Exemplo: Troca de Chaves Diffie-Hellman

A segurança da troca de chaves Diffie-Hellman está no fato de que, embora seja fácil calcular exponenciais módulo um primo, é muito difícil calcular logaritmos discretos. Para números grandes, esta tarefa é inviável.

Exemplo: $q = 353$, $\alpha = 3$ $X_A = 97$, $X_B = 233$

Cálculo das chaves públicas:

$$Y_A = 3^{97} \bmod 353 = 40 \quad Y_B = 3^{233} \bmod 353 = 248$$

Cálculo da chave secreta comum:

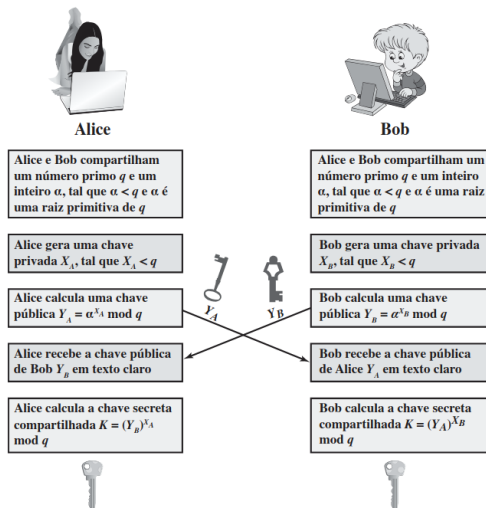
$$K_A = (Y_B)^{X_A} \bmod 353 = 248^{97} \bmod 353 = 160$$

$$K_B = (Y_A)^{X_B} \bmod 353 = 40^{233} \bmod 353 = 160$$

Para um intruso: Disponível apenas $q = 353$, $\alpha = 3$, $Y_A = 40$, $Y_B = 248$. Para descobrir K , teria de resolver um *logaritmo discreto*, o que é inviável para primos grandes.

Diffie-Hellman na prática

Figura 10.1 O algoritmo de troca de chaves Diffie-Hellman.



Troca simples (dois usuários):

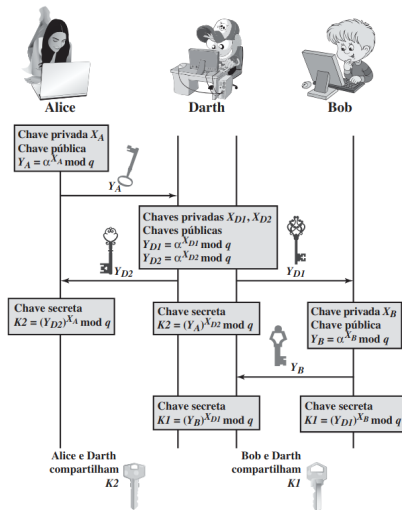
- A escolhe X_A , calcula $Y_A = \alpha^{X_A} \bmod q$ e envia a B .
- B escolhe X_B , calcula $Y_B = \alpha^{X_B} \bmod q$ e envia a A .
- Ambos calculam a chave secreta K :

$$K = (Y_B)^{X_A} \bmod q = (Y_A)^{X_B} \bmod q$$

- Os valores públicos q e α devem ser conhecidos previamente ou enviados na primeira mensagem.

Man-in-the-Middle - Diffie Hellman

Figura 10.2 Ataque *man-in-the-middle*.



Ataque Man-in-the-Middle em Diffie–Hellman

Consequência:

- Alice e Bob acreditam ter uma chave secreta compartilhada.
- Na prática, Alice compartilha K_2 com Darth e Bob compartilha K_1 com Darth.
- Darth pode **espiar** ou **modificar** mensagens.

Motivo da vulnerabilidade: O protocolo não autentica os participantes.

Solução: uso de **assinaturas digitais** e **certificados**.

Aula 12 - IPsec

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
June 2, 2026

IPsec: O Protocolo IP de Segurança

- Provê segurança na camada de rede.
- Protege datagramas IP entre quaisquer entidades (dispositivos finais (*hosts*), roteadores).
- Usado para criar Redes Privadas Virtuais (VPNs) sobre a Internet pública.
- Definido pelo IAB como essencial para o IPv6 (autenticação e criptografia).
- Projetado para ser compatível tanto com IPv4 quanto com IPv6.
- Atualmente, muitos fabricantes já oferecem suporte ao IPsec.

Relembrando encapsulamento em redes

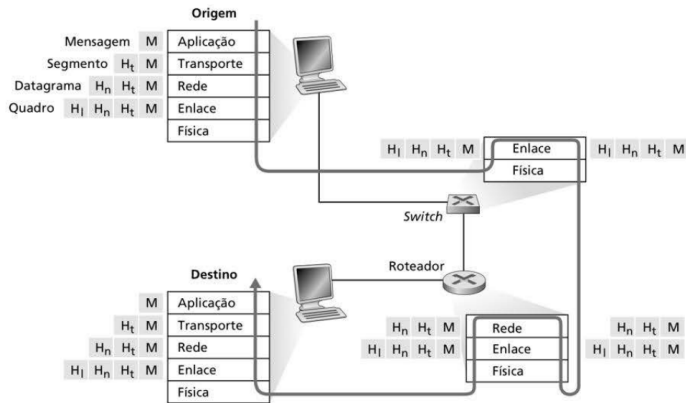


Figura 1.24 Hospedeiros, roteadores e switches; cada um contém um conjunto diferente de camadas, refletindo suas diferenças em funcionalidade.

Relembrando o formato do datagrama IPv4

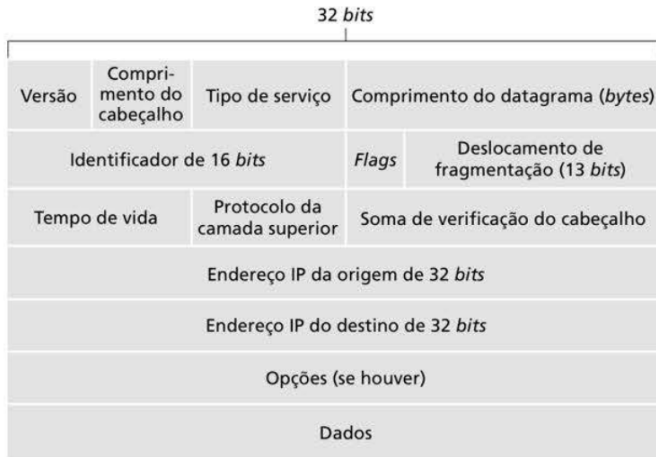


Figura 4.17 Formato do datagrama IPv4.

Áreas Funcionais da Segurança em Nível IP

- A segurança em nível IP abrange três áreas funcionais:

1. Autenticação

- Garante que o pacote foi realmente enviado pela fonte identificada.
- Assegura que o pacote não foi alterado durante o trânsito.

2. Confidencialidade

- Permite aos nós da rede cifrar (criptografar) mensagens.
- Previne a espionagem (eavesdropping) por terceiros.

3. Gerenciamento de Chaves

- Trata da troca segura de chaves entre as partes.

O Conceito de Sigilo na Camada de Rede

- O que significa prover sigilo na camada de rede?
 - A entidade remetente (hospedeiro ou roteador) cifra as **cargas úteis (payloads)** de todos os datagramas que envia.
- O que é a carga útil (payload)?
 - Segmento TCP
 - Segmento UDP
 - Mensagem ICMP
 - Mensagem SNMP, etc.
- Resultado: "**Cobertura Total**"
 - Todos os dados (e-mails, páginas Web, mensagens de gerenciamento) ficam ocultos de intrusos.

Além da confidencialidade, um protocolo de segurança da camada de rede pode prover:

- **Autenticação da Origem**
 - Permite à entidade destinatária verificar a origem (remetente) do datagrama protegido.
- **Integridade dos Dados**
 - Permite à entidade destinatária verificar se o datagrama sofreu qualquer adulteração durante o trânsito.
- **Prevenção do Ataque de Repetição (Replay Attack)**
 - Permite ao destinatário (Bob) detectar quaisquer datagramas duplicados que um atacante possa inserir no fluxo.
- **Característica:** IPsec pode **criptografar e/ou autenticar todo o tráfego no nível IP.**

- O IPsec pode ser usado em LANs, WANs (públicas/privadas) e na Internet.
- Todas as aplicações distribuídas se beneficiam, sem necessidade de modificação.
- **Exemplos de uso:**
 - **Conectividade Segura de Filiais (VPN):**
 - Permite construir uma VPN segura sobre a Internet.
 - Reduz custos e a necessidade de redes privadas.
 - **Acesso Remoto Seguro:**
 - Usuários remotos (viajantes, telecommuters) acessam a rede da empresa de forma segura através de um ISP local.
 - **Conectividade Extranet/Intranet com Parceiros:**
 - Protege a comunicação com outras organizações (autenticação, confidencialidade, troca de chaves).
 - **Reforço à Segurança de E-commerce:**
 - Adiciona segurança, mesmo que as aplicações Web já possuam protocolos de segurança próprios.

Benefícios do IPsec

- **Implementação em Firewall/Roteador**
 - Aplica segurança forte a todo o tráfego que cruza o perímetro.
 - O tráfego interno (workgroup) não sofre overhead de processamento.
- **Resistência a Bypass**
 - Difícil de contornar (bypass) se o firewall IPsec for a única entrada da Internet.
- **Transparência para Aplicações**
 - Atua abaixo da camada de transporte (TCP/UDP).
 - Não exige mudança de software em usuários ou servidores.
 - Aplicações de camada superior não são afetadas.
- **Transparência para o Usuário Final**
 - Não é necessário treinar os usuários sobre os mecanismos de segurança.
 - Não é preciso gerenciar chaves por usuário (emitir/revogar).
- **Flexibilidade (Segurança Individual)**
 - Pode prover segurança para usuários individuais, se necessário.
 - Útil para trabalhadores remotos (off-site).
 - Permite criar sub-redes virtuais seguras para aplicações sensíveis.

IPsec e Redes Privadas Virtuais (VPNs)

- **O Cenário:**

- Instituições que se estendem por diversas regiões geográficas.

- **A Necessidade:**

- Desejam ter sua própria rede IP.
- Objetivo: Permitir que seus hóspedes e servidores enviem dados uns aos outros de forma segura e sigilosa.

- **A Solução Tradicional: Rede Privada**

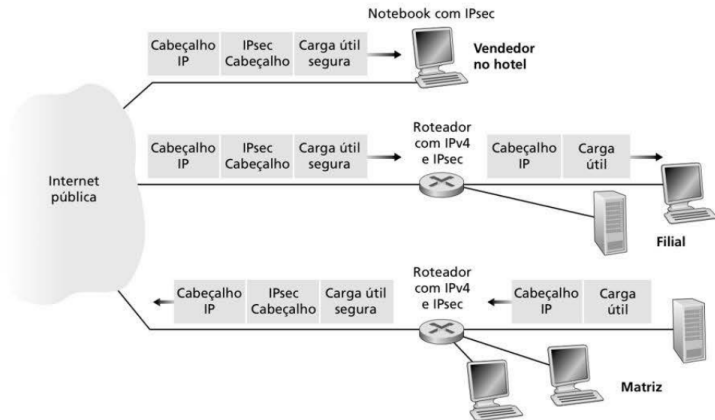
- Definição: Uma rede física independente, reservada a uma instituição particular.
- Características:
- É completamente separada da Internet pública.
- Requer infraestrutura própria: roteadores, enlaces e DNS.

- **O Problema da Rede Privada:**

- Custo muito elevado.
- A instituição precisa comprar, instalar e manter sua própria infraestrutura de rede física.

A Solução: Rede Privada Virtual (VPN)

- VPNs operam "em cima" (sobre) da Internet pública.
- O tráfego é **criptografado** e enviado através da infraestrutura da Internet pública.
- Não é necessária uma rede fisicamente independente e dedicada.
- A criptografia é aplicada **antes** que os pacotes entrem na Internet pública.



VPN: Cenário de Exemplo e Fluxo de Tráfego

- **Componentes da Instituição (Exemplo):**
 - Matriz, Filial e outros acessando da Internet
- **Fluxo 1: Comunicação Interna (Dentro de um Local)**
 - Ex: Hospedeiro na Matriz ↔ Hospedeiro na Matriz.
 - Ex: Hospedeiro na Filial ↔ Hospedeiro na Filial.
 - Método: IP v4 padrão (sem IPsec). O tráfego não sai para a rede pública.
- **Fluxo 2: Comunicação Externa (Entre Locais via VPN)**
 - Ex: Hospedeiro na Matriz ↔ Hospedeiro na Filial.
 - Ex: Vendedor Viajante ↔ Hospedeiro na Matriz.
 - Caminho: O tráfego deve cruzar a Internet pública.
 - Método: O tráfego é **codificado (criptografado)** antes de entrar na Internet (usando IPsec).

- **Ponto Importante:**

- Nem todo o tráfego enviado à Internet pelos roteadores de borda ou pelos notebooks será protegido por IPsec.

- **Exemplo: Acesso à Internet Pública**

- Um hospedeiro na matriz quer acessar um servidor Web comum (ex: Amazon, Google).
- Este tráfego não é destinado à VPN.

- **Resultado: Coexistência de Tráfego**

- O roteador de borda (e os notebooks) emitirão **ambos** os tipos de datagramas para a Internet:
 - Datagramas IPv4 comuns (para tráfego público).
 - Datagramas IPsec seguros (para tráfego da VPN).

VPN: Fluxo de Tráfego (Matriz → Vendedor Viajante)

- **1. Na Matriz (Origem)**
 - Um hospedeiro envia um datagrama IPv4 padrão (ex: TCP/UDP).
- **2. No Roteador de Borda (Gateway IPsec da Matriz)**
 - Intercepta o datagrama IPv4.
 - Converte o datagrama IPv4 em um datagrama **IPsec**.
 - Encaminha o pacote IPsec para a Internet.
- **3. Na Internet (Trânsito)**
 - O datagrama IPsec possui um cabeçalho IPv4 tradicional (externo).
 - Roteadores da Internet o processam como um datagrama IPv4 comum.
- **4. Estrutura do Pacote IPsec (no Trânsito)**
 - A carga útil (payload) do datagrama IPsec contém:
 - Um **cabeçalho IPsec** (para processamento de segurança).
 - A carga útil original (ex: segmento TCP/UDP), que está **codificada** (criptografada).
- **5. No Notebook do Vendedor (Destino)**
 - O S.O. recebe o datagrama IPsec.
 - **Decodifica** a carga útil (descriptografa).
 - Verifica outros serviços (ex: integridade dos dados).
 - Passa a carga útil original (não codificada) para a camada superior (TCP ou UDP).

Protocolos Principais do IPsec: AH vs. ESP

O conjunto IPsec possui dois protocolos principais para envio de dados seguros (entre hospedeiros ou roteadores):

- **1. Protocolo Cabeçalho de Autenticação (AH - Authentication Header)**
 - **Provê** Autenticação da Origem.
 - **Provê** Integridade dos Dados.
 - **NÃO** provê Sigilo (Confidencialidade).
- **2. Protocolo Carga de Segurança de Encapsulamento (ESP - Encapsulating Security Payload)**
 - **Provê** Autenticação da Origem.
 - **Provê** Integridade dos Dados.
 - **Provê Sigilo (Confidencialidade)** ← (Função combinada de autenticação/criptografia).

O Foco no ESP (AH é Deprecated)

- **Por que o ESP é muito mais utilizado?**
 - O Sigilo (Criptografia) é muitas vezes essencial para VPNs.
 - VPNs geralmente desejam **autenticação E criptografia** para:
 - 1. Impedir que usuários não autorizados penetrem na rede.
 - 2. Impedir que espiões (eavesdroppers) na Internet leiam as mensagens.
- **O Status Atual do AH: é deprecated (obsoleto)**
 - Motivo: O próprio protocolo ESP já fornece a autenticação da mensagem.
 - O AH é mantido no IPsecv3 apenas para *backward compatibility* (compatibilidade retroativa).
 - **Não deve ser usado em novas aplicações.**
- **Foco atual: ESP.**

Associações de Segurança (SA)

- Datagramas IPsec são enviados entre pares de entidades da rede (hospedeiro ↔ hospedeiro, roteador ↔ roteador, etc.).
- **O que é uma Associação de Segurança (SA - Security Association)?**
 - É uma **conexão lógica** da camada de rede.
 - Criada **antes** que a entidade remetente possa enviar datagramas IPsec à entidade destinatária.
- **Característica Chave: Unidirecional**
 - Uma SA é uma conexão lógica **simples** (simplex).
 - Ela flui em apenas uma direção: do remetente para o destinatário.
- **Comunicação Bidirecional (Mútua)**
 - Se duas entidades querem enviar dados seguros entre si (nos dois sentidos): Precisam ser estabelecidas **duas SAs** (duas conexões lógicas).
 - Uma SA em cada direção.

Exemplo: Contagem de SAs em uma VPN

- **Cenário (Instituição):**
 - 1 Matriz.
 - 1 Filial.
 - n Vendedores Viajantes (notebooks).
- **Fluxos de Tráfego IPsec (Bidirecional):**
 - Tráfego bidirecional entre o roteador de borda da Matriz e o roteador de borda da Filial.
 - Tráfego bidirecional entre o roteador de borda da Matriz e CADA UM dos n Vendedores.
- **Cálculo - Quantidade de SA:**
 - **Matriz $\leftrightarrow$ Filial:**
 - 1 SA (Matriz $\rightarrow$ Filial)
 - 1 SA (Filial $\rightarrow$ Matriz)
 - *Subtotal* = 2 SAs
 - **Matriz $\leftrightarrow n$ Vendedores:**
 - Para cada vendedor, são necessárias 2 SAs (uma em cada direção).
 - *Subtotal* = $2 \times n$ SAs
- **Total de SAs na VPN:**
 - Total = (SAs da Filial) + (SAs dos Vendedores)
 - Total = $2 + 2n$ SAs

O Banco de Dados de Associações de Segurança (SAD)

- Toda implementação IPsec inclui um **Banco de Dados de Associações de Segurança** (SAD - Security Association Database).
- Este banco de dados armazena e define todos os parâmetros associados a cada SA (Associação de Segurança) ativa.
- Uma SA é caracterizada por um conjunto de parâmetros específicos. **Que vamos ver nos próximos slides**
- Uma entidade IPsec (roteador ou hospedeiro) armazena informações de estado para **muitas SAs** simultaneamente.
- **Relembrando o Exemplo da VPN:**
 - Cenário: 1 Matriz, 1 Filial e n vendedores.
 - O roteador de borda da Matriz guarda informações de estado para $(2 + 2n)$ SAs.

O SPD - Security Policy Database

O Problema: R1 recebe um datagrama IPv4 da rede interna destinado a um IP de fora.

- **Como R1 sabe** se este datagrama deve ser convertido em IPsec (ou enviado como IPv4 comum)?
 - ... e se for IPsec, qual SA (dentre as várias no SAD) ele deve usar?
- **A Solução: O Banco de Dados de Política de Segurança (SPD)**
 - A entidade IPsec (R1) mantém esta *outra* estrutura de dados (além do SAD).
 - O SPD indica **quais tipos** de datagramas serão processados pelo IPsec.
 - A decisão é baseada em: Endereço IP remetente, Endereço IP destinatário, e Tipo do protocolo (TCP/UDP).
 - O SPD também aponta para qual SA deve ser usada.
- **A Diferença Chave: SPD vs. SAD**
 - **SPD (Política):** Indica "**O QUE**" fazer com um datagrama (Processar com IPsec, Descartar, Deixar passar).
 - **SAD (Associação):** Indica "**COMO**" fazer o IPsec (Chaves de criptografia/autenticação, algoritmos, SPI, etc).

Informações de Estado da SA (Exemplo: Roteador R1)

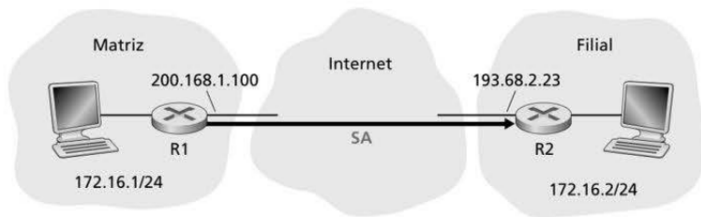


Figura 8.28 Associação de segurança (SA) de R1 a R2.

Uso das Informações de Estado da SA (Exemplo R1 → R2)

- **No Remetente (Roteador R1)**

- Quando R1 precisa construir um datagrama IPsec para encaminhar por essa SA:
- Ele acessa as informações de estado (SPI, chaves, algoritmos) da SA.
- Usa essas informações para determinar como deve **autenticar** e **criptografar** o datagrama.

- **No Destinatário (Roteador R2)**

- R2 mantém as *mesmas* informações de estado sobre essa SA (identificada pelo SPI).
- Ele as usa para **autenticar** (verificar a integridade) e **decodificar** (descriptografar) qualquer datagrama IPsec que chegar da SA.

Informações de Estado da SA (Exemplo: Roteador R1)

O Roteador R1 manterá as informações de estado sobre essa SA:

- Um identificador de 32 bits para a SA: **Índice de Parâmetro de Segurança (SPI)**
- As interfaces da SA:
 - Interface remetente (Origem): 200.168.1.100
 - Interface destinatária (Destino): 193.68.2.23
- Parâmetros de Criptografia (para Sigilo):
 - O tipo de criptografia (ex: 3DES com CBC).
 - A chave de criptografia.
- Parâmetros de Integridade (para Autenticação):
 - O tipo de verificação de integridade (ex: HMAC com MD5).
 - A chave de autenticação.

Parâmetros da SA (Parte 1)

- **Contador de Número de Sequência (Sequence number counter)**
 - Um valor de 32 bits usado para gerar o campo "Sequence Number" nos cabeçalhos AH ou ESP
 - Essencial para o antireplay.
- **Estouro do Contador (Sequence counter overflow)**
 - Uma "flag" que indica se o estouro (overflow) do contador deve:
 - Gerar um evento auditável (log).
 - Impedir a transmissão de mais pacotes nesta SA.
- **Janela Antireplay (Antireplay window)**
 - Define uma "janela deslizante" (sliding window).
 - Usada para determinar se um pacote AH ou ESP recebido é um *replay* (pacote duplicado/malicioso).
 - O número de sequência do pacote deve estar dentro desta janela.

Parâmetros da SA (Parte 2)

- **Informações ESP**

- Algoritmo de criptografia e autenticação.
- Chaves, Valores de Inicialização (IVs), tempos de vida das chaves e parâmetros relacionados ao ESP.

- **Tempo de Vida da SA (Lifetime)**

- Um intervalo de tempo ou uma contagem de bytes.
- Após atingido, a SA deve ser substituída por uma nova SA (e novo SPI) ou terminada.

- **Modo do Protocolo IPsec (Protocol mode)**

- Define o modo de operação: Túnel (Tunnel) ou Transporte (Transport)

- **Path MTU**

- Armazena o "Maximum Transmission Unit" (tamanho máximo do pacote) observado no caminho.
- Usado para evitar fragmentação de pacotes.
- Inclui variáveis de "envelhecimento" (aging).

O Datagrama IPsec: Modo Túnel vs. Modo Transporte

- Analisando o datagrama IPsec.
- O IPsec possui **duas formas diferentes de pacotes** (dois modos de operação):
 - **Modo Túnel (Tunnel Mode)**
 - **Modo Transporte (Transport Mode)**
- **Comparação de Uso:**
 - O **Modo Túnel** é o mais apropriado para o cenário de VPNs (que vimos anteriormente).
 - Por esta razão, o Modo Túnel é **mais implementado** do que o Modo Transporte.
- **Foco da Aula:**
 - A partir de agora vamos nos concentrar **exclusivamente no Modo Túnel**.
 - (Uma vez que se tem uma compreensão sólida do Modo Túnel, é fácil aprender o Modo Transporte).

Processo de Criação do Datagrama ESP (Modo Túnel) - Parte 1

O processo de encapsulamento (ex: no roteador de borda) segue estes passos

- **1. Criptografia e criação da mensagem**

- Pega-se o datagrama IP original (que será protegido).
- Criptografa-se este datagrama (ele se torna o campo "Payload Data").
- Anexa-se na frente dele o "Cabeçalho ESP" (SPI + Seq. Number).
- O pacote resultante é o Cabeçalho ESP + Carga Criptografada + Trailers ESP.

- **2. Autenticação (MAC/ICV)**

- Cria-se uma autenticação MAC (ou ICV - Integrity Check Value) sobre o datagrama inteiro.
- Utiliza-se o algoritmo (ex: HMAC) e a chave de autenticação especificados na SA.

- **3. Formação da Carga Útil (Payload) ESP**
 - Anexa-se o MAC (ICV) atrás do datagrama.
 - O resultado (Cabeçalho ESP + Carga Criptografada + Trailers + MAC) forma a **Carga Útil ESP** completa.
- **4. Criação do Novo Cabeçalho IP (Modo Túnel)**
 - Por fim, cria-se um **novo cabeçalho IP** (IPv4 clássico, 20 bytes).
 - Este novo cabeçalho é anexado **antes** da Carga Útil ESP.
 - Este é o cabeçalho que os roteadores da Internet irão ler.

Datagrama IPsec

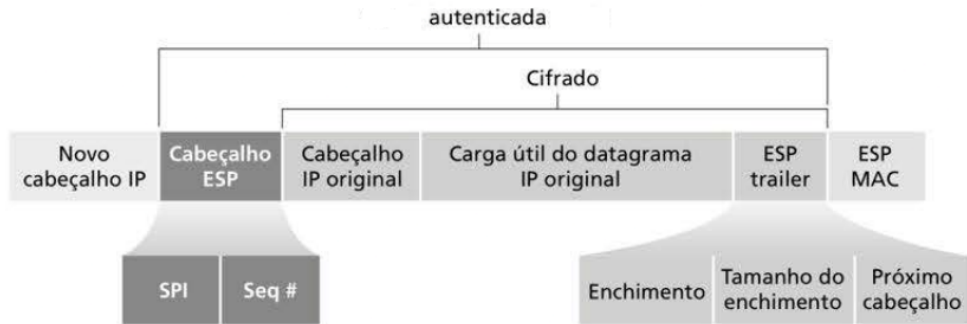


Figura 8.29 Formato do datagrama IPsec.

O Datagrama IPsec Resultante (Modo Túnel)

- **A Carga Útil (Payload) deste datagrama contém:**
 - O Cabeçalho ESP (SPI, Seq. Number).
 - O **Datagrama IP Original** (cifrado).
 - O Trailer ESP (Padding, Pad Length, Next Header) (cifrado).
 - O Campo de Autenticação ESP (ICV/MAC).
- **Cabeçalho IP Original (Interno e Cifrado)**
 - Contém os endereços dos *hospedeiros finais*.
 - Endereço IP Remetente (Ex): 172.16.1.17
 - Endereço IP Destinatário (Ex): 172.16.2.48
 - Estes endereços são **cifrados** e invisíveis para a Internet.
- **Novo Cabeçalho IP (Externo e Visível)**
 - Contém os endereços das *extremidades do túnel* (os gateways IPsec).
 - Endereço IP Remetente (Ex): 200.168.1.100
 - Endereço IP Destinatário (Ex): 193.68.2.23
 - O campo "Protocolo" deste novo cabeçalho é **50** (indicando ESP), e não TCP(6) ou UDP(17).

Revisão: O Cabeçalho ESP (Enviado em Aberto)

- O "Cabeçalho ESP" é anexado na frente da unidade cifrada (payload + trailer).
- Este cabeçalho **não é criptografado** (é enviado "em aberto" / "clear text").
- Ele consiste em dois campos principais:
 - **1. O Índice de Parâmetro de Segurança (SPI)**
 - **Função:** Indica à entidade destinatária (ex: R2) a qual Associação de Segurança (SA) este datagrama pertence.
 - **Ação no Destino:** A entidade usa o SPI para indexar (localizar) rapidamente a SA correta em seu Banco de Dados (SAD).
 - Ao localizar a SA, o destino sabe quais algoritmos e chaves deve usar para autenticar e decriptar o pacote.
 - **2. O Campo de Número de Sequência**
 - **Função:** É usado para a proteção contra ataques de repetição (replay attacks).
 - (O destinatário verifica este número contra a sua "janela antireplay" definida na SA).

O Trailer ESP: Campos e Criptografia

- O trailer ESP consiste em três campos:
- **Importante:** O trailer é adicionado ao datagrama IP original **antes** da cifragem.
- **1. Enchimento (Padding)**
 - **O quê:** Bytes sem significado.
 - **Por quê:** Cifras de bloco exigem que a mensagem a ser cifrada seja um múltiplo inteiro do comprimento de bloco (ex: 128 bits para o AES).
 - O enchimento é usado para "completar" o bloco.
- **2. Tamanho do Enchimento (Pad Length)**
 - **O quê:** Um campo que indica quantos bytes de enchimento (*padding*) foram inseridos.
 - **Por quê:** Permite à entidade destinatária saber exatamente quanto enchimento deve remover após a decifração.
- **3. Próximo Cabeçalho (Next Header)**
 - **O quê:** Identifica o tipo de dados da carga útil (o datagrama original).
 - **Por quê:** Informa ao S.O. do destinatário para qual protocolo entregar o pacote decifrado (ex: TCP, UDP, ICMP).
- **Resumo do Processo de Cifragem:**
 - Os [Dados da Carga Útil (Datagrama IP original)] e o [Trailer ESP] são concatenados.
 - O resultado concatenado é então **cifrado** como uma única unidade.

O MAC de Autenticação ESP (ICV)

- Além da criptografia, a entidade remetente anexa uma **Autenticação MAC** (MAC - Message Authentication Code) ou ICV.
- **Sobre o que o MAC é calculado?**
 - O remetente calcula o MAC sobre **toda o datagrama**.
- **O que o cálculo do MAC?**
 - O Cabeçalho ESP (SPI + Seq. Number) - (em aberto).
 - O Datagrama IP original (já criptografado).
 - O Trailer ESP (Padding, etc.) (já criptografado).
- **Como o MAC é calculado? (Revisão)**
 - O remetente anexa uma **chave MAC secreta** definida na SA.
 - Em seguida, calcula um **hash de tamanho fixo** do resultado (ex: HMAC-MD5 ou HMAC-SHA1).
 - Este hash é o MAC/ICV, que é anexado ao final do pacote.

Processamento no Destino (Exemplo: Roteador R2)

- Quando R2 recebe o datagrama IPsec (destino: R2, protocolo: 50):
- **1. Identificação da SA:**
 - R2 entende que é um pacote ESP (Protocolo 50).
 - Ele lê o campo **SPI** (no cabeçalho ESP) para determinar a qual SA o datagrama pertence (procurando no SAD).
- **2. Verificação de Autenticidade/Integridade:**
 - R2 calcula o MAC sobre o datagrama (usando a chave da SA).
 - Compara o MAC calculado com o valor no campo MAC ESP.
 - Se forem compatíveis: R2 sabe que (A) o pacote veio de R1 e (B) não foi adulterado.
- **3. Verificação Antireplay:**
 - R2 verifica o campo **Número de Sequência** para garantir que o datagrama é novo (e não um pacote repetido).

- **4. Decodificação (Descriptografia):**
 - R2 decodifica a unidade cifrada (payload + trailer) usando o algoritmo e a chave de criptografia da SA.
- **5. Extração do Pacote Original:**
 - R2 remove o enchimento (padding) e extrai o datagrama IP original (que estava cifrado).
- **6. Encaminhamento:**
 - R2 finalmente encaminha o datagrama IP original (agora em claro) para a rede da filial, em direção ao seu destino final (ex: 172.16.2.48).

- **O Desafio: Como criar as Associações de Segurança (SAs)?**
- **Opção 1: "Teclagem Manual" (Manual Keying)**
 - **Cenário:** VPNs com um número pequeno de pontos finais (ex: 2 roteadores).
 - **Ação:** O administrador de rede pode inserir *manualmente* as informações da SA (algoritmos, chaves, SPIs) nos SADs das entidades.
- **O Problema desta Abordagem:**
 - É claramente **impraticável** para uma VPN grande.
 - (Ex: Redes com centenas ou milhares de roteadores e hospedeiros IPsec).
- **Opção 2: O Mecanismo Automático (IKE)**
 - Implementações grandes e geograficamente distribuídas exigem um mecanismo automático para a criação das SAs.
 - O IPsec utiliza para isso o protocolo **IKE (Internet Key Exchange)**.
 - Especificado na **RFC 5996**.

IKE: Funções Principais do Protocolo

- O protocolo IKE (Internet Key Exchange) é o mecanismo automático para criar SAs.
- **Responsabilidades do IKE:**
 - **1. Autenticação das Entidades**
 - Troca certificados para provar a identidade das partes (ex: R1 e R2).
 - **2. Negociação de Parâmetros**
 - Negocia os algoritmos de criptografia (ex: AES, 3DES).
 - Negocia os algoritmos de autenticação (ex: HMAC-SHA1).
 - **3. Geração de Chaves**
 - Troca informações de chave com segurança (usando Diffie-Hellman).
 - Cria as chaves de sessão (secretas) que serão usadas nas SAs do IPsec.

IKE: A Estrutura de Duas Fases

- O IKE opera em duas fases principais (contexto: R1 e R2).
- **Fase 1: O Canal Seguro**
 - Objetivo: Criar um canal seguro e autenticado **para o próprio IKE**.
 - Consiste em duas trocas de pares de mensagens.
 - Resultado: Criação de uma **IKE SA** (bidirecional).
- **Atenção: IKE SA $\neq$ IPsec SA**
 - A **IKE SA** (Fase 1) é um canal seguro para proteger as negociações do IKE.
 - As **IPsec SAs** (Fase 2) são as conexões unidirecionais usadas para proteger os dados do usuário (o tráfego da VPN).

- **Primeira Troca de Mensagens (Anônima)**
 - Os dois lados (R1, R2) usam **Diffie-Hellman**.
 - **Objetivo:** Criar a IKE SA bidirecional.
 - Chaves são estabelecidas para criptografia e autenticação **da IKE SA**.
 - Um **segredo mestre** é estabelecido (para ser usado na Fase 2).
 - *Importante:* Nesta etapa, nem R1 nem R2 revelam sua identidade (não assinam nada com chaves privadas).
- **Segunda Troca de Mensagens (Autenticada)**
 - Ambos os lados **revelam suas identidades** (ex: certificados) assinando as mensagens.
 - *Segurança:* As identidades não são expostas a "analísadores passivos", pois esta troca já ocorre **dentro do canal seguro IKE SA** criado na primeira troca.
 - *Negociação:* Os dois lados negociam os algoritmos de autenticação e criptografia que serão usados pelas futuras **SAs IPsec** (da Fase 2).

- **IKE Fase 2: Criação das SAs IPsec**

- Ao fim da Fase 1, existe um canal IKE SA seguro e autenticado.
- Na Fase 2, os dois lados usam este canal para criar as **SAs do IPsec**.
- Criam uma SA IPsec em cada direção (duas SAs unidirecionais).
- As chaves de sessão de criptografia e autenticação são estabelecidas para ambas as SAs IPsec.
- Agora, podem usar essas SAs IPsec para enviar dados seguros.

- **A Motivação para as Duas Fases**

- **Custo Computacional.**
- A Fase 1 é "cara" (envolve criptografia de chave pública: Diffie-Hellman e assinaturas RSA).
- A Fase 2 é "barata" (não envolve chave pública; usa o segredo mestre da Fase 1).
- **Vantagem:** Permite criar um *grande número* de SAs IPsec (Fase 2) com um custo computacional relativamente pequeno, após estabelecer apenas uma IKE SA (Fase 1).

Aula 13 - TLS

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
June 2, 2026

Protegendo conexões TCP: TLS

- **Contextualização:**
 - Vimos segurança na camada de rede.
 - Agora, subimos uma camada na pilha de protocolo.
- **O Objetivo do TLS:**
 - Examinar como a criptografia pode aprimorar o **TCP**.
 - (Transport Layer Security).
- **Serviços de Segurança Adicionados ao TCP:**
 - Sigilo (Confidencialidade).
 - Integridade dos Dados.
 - Autenticação do Ponto Final (End-point Authentication).
- **Padronização e Histórico:**
 - O TLS é padronizado pelo IETF na **RFC 4346**.
 - Uma versão anterior e semelhante deste protocolo é o **SSL (Secure Sockets Layer)** versão 3.

TLS: Histórico e Adoção

- **Origem:** O SSL (Secure Sockets Layer) foi originalmente projetado pela Netscape.
 - (Ideias básicas sobre proteção do TCP antecedem este trabalho, ex: Woo, 1994).
- **Sucessor:** O TLS (Transport Layer Security) é o sucessor padronizado do SSL.
- **Adoção Massiva:**
 - Suportado por todos os navegadores Web e servidores Web populares.
 - Usado pelo Gmail e por **basicamente todos os sites de comércio eletrônico** (Amazon, eBay, TaoBao).
 - Centenas de bilhões de dólares são gastos anualmente usando TLS.
- **Como o usuário identifica o TLS?**
 - Se você já comprou algo online com cartão de crédito, sua comunicação foi quase certamente via TLS.
 - A URL no navegador se inicia com **https://** (em vez de http://).

A Necessidade do TLS: Cenário de E-commerce

- **O Cenário Típico:**

- **Bob** (cliente) está navegando na Web.
- Ele acessa o site da **Alice Incorporated** (vendedor) para comprar perfume.

- **A Ação:**

- O site exibe um formulário.
- Bob insere informações sensíveis:
 - Tipo de perfume e quantidade.
 - Seu endereço.
 - O **número de seu cartão de crédito**.
- Bob clica em "Enviar".

- **A Expectativa (sem segurança):**

- Bob espera receber o perfume e a cobrança correta.
- **O Risco:** Se nenhuma medida de segurança for tomada, Bob pode ter "surpresas".

Riscos: O Cenário de E-commerce sem TLS

- **1. Se nenhuma Confidencialidade (Sigilo) for utilizada:**
 - Um invasor (Trudy) poderia **interceptar** o pedido.
 - Trudy rouba as informações do cartão de crédito de Bob.
 - Consequência: O invasor pode fazer compras à custa de Bob.
- **2. Se nenhuma Integridade dos Dados for utilizada:**
 - Um invasor poderia **modificar** o pedido de Bob em trânsito.
 - Ex: Fazer Bob comprar 10x mais frascos de perfume do que o desejado.
- **3. Se nenhuma Autenticação do Servidor for utilizada:**
 - Um servidor (mantido por Trudy) poderia **se passar** pela Alice Inc., usando o mesmo logotipo.
 - Bob insere seus dados no site falso de Trudy.
 - Consequência: Trudy fica com o dinheiro e some, ou pratica **roubo de identidade** (nome, endereço, cartão).

TLS: Além do HTTP e Posição na Pilha

- **Uso Comum: HTTPS**

- Muitas vezes, o TLS é usado para oferecer segurança em transações que ocorrem pelo HTTP.

- **O Verdadeiro Alcance: Qualquer Aplicação TCP**

- Como o TLS protege o **TCP**, ele pode ser empregado por **qualquer aplicação** que execute sobre o TCP (ex: FTP, SMTP, etc.).

- **Interface para o Desenvolvedor (API)**

- O TLS provê uma Interface de Programação de Aplicações (API) com Sockets, muito semelhante à API de Sockets do TCP.
- Para usar o TLS, a aplicação simplesmente inclui as classes ou bibliotecas SSL/TLS.

- **Localização na Pilha de Protocolos**

- Tecnicamente, o TLS reside na **Camada de Aplicação**.
- Do ponto de vista do **desenvolvedor**, ele é visto como um **protocolo de transporte** que provê serviços do TCP aprimorados com serviços de segurança.
- (Conforme Figura 8.24: TLS fica entre a Aplicação e o TCP).

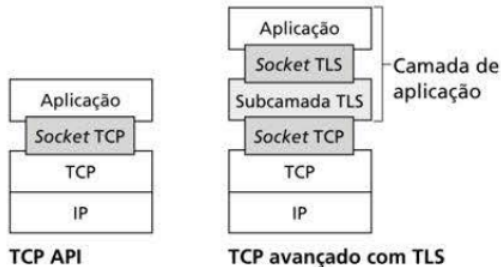


Figura 8.24 Embora o TLS resida tecnicamente na camada de aplicação, do ponto de vista do desenvolvedor, ele é um protocolo da camada transporte.

Arquitetura do TLS

- O TLS é projetado para **usar o TCP** para prover um serviço seguro, confiável (reliable) e fim-a-fim.
- O TLS não é um protocolo único, mas sim **duas camadas** de protocolos.
- **Camada 1: O Protocolo de Registro (Record Protocol)**
 - Fica "em cima" do TCP.
 - Fornece os serviços básicos de segurança (criptografia, integridade) para os protocolos da camada superior.
- **Camada 2: Protocolos Superiores (Gerenciamento TLS)**
 - São protocolos **específicos do TLS**, usados na **gestão** das trocas TLS.
 - São três protocolos que rodam "em cima" do Record Protocol:
 - Protocolo Handshake (Handshake Protocol)
 - Protocolo de Troca de Especificação de Cifra (Change Cipher Spec Protocol)
 - Protocolo de Alerta (Alert Protocol)
- **Camada de Aplicação (Exemplo)**
 - O **HTTP** (Hypertext Transfer Protocol), usado para interação Web cliente/servidor, pode operar "em cima" do TLS.

A pilha SSL/TLS

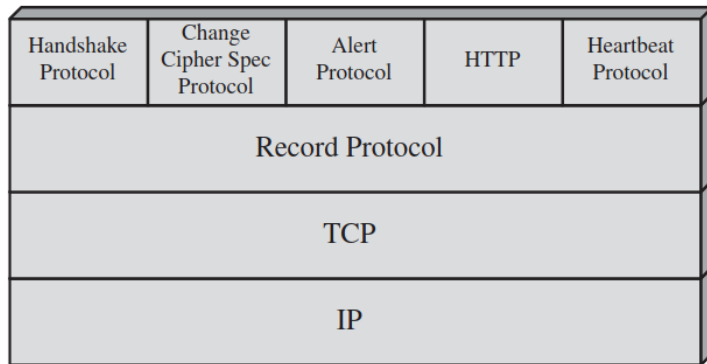


Figure 22.4 SSL/TLS Protocol Stack

Conceitos Importantes: Conexão TLS vs. Sessão TLS

- Existem dois conceitos centrais: Conexão e Sessão.
- **Conexão (Connection)**
 - É um transporte (no sentido OSI) que provê um serviço.
 - No TLS, é uma relação *peer-to-peer*.
 - As conexões são **transitórias** (passageiras).
 - **Toda conexão está associada a UMA sessão.**
- **Sessão (Session)**
 - É uma associação entre um cliente e um servidor.
 - Criada pelo **Handshake Protocol**.
 - Define um conjunto de **parâmetros de segurança criptográficos** (chaves, algoritmos, etc.).
 - Estes parâmetros podem ser **compartilhados** entre múltiplas conexões.
 - **Objetivo:** Evitar a negociação (cara) de novos parâmetros de segurança para cada nova conexão.
- **Relacionamento:**
 - Pode haver **múltiplas conexões seguras** entre um cliente e um servidor.
 - Na prática, essas múltiplas conexões compartilham (reutilizam) os parâmetros de uma **única sessão**.

O protocolo de registro do TLS

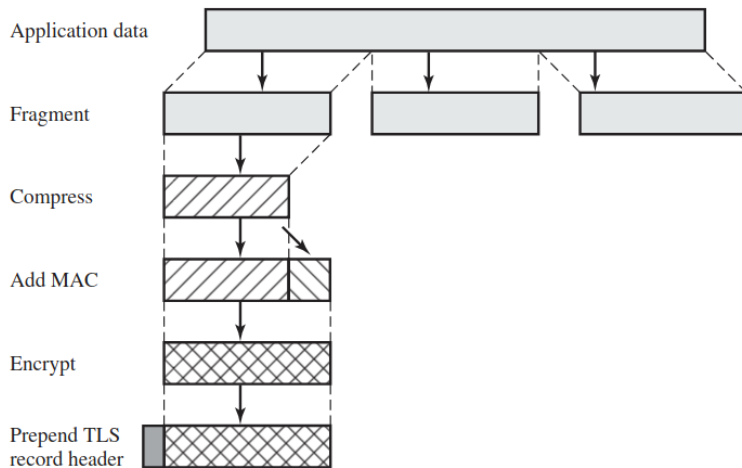


Figure 22.5 TLS Record Protocol Operation

Protocolo de Registro TLS (Record Protocol): Serviços

- O Protocolo de Registro (SSL/TLS Record Protocol) provê dois serviços básicos para as conexões TLS:
- **1. Confidencialidade (Sigilo)**
 - O Protocolo Handshake (visto mais tarde) define uma **chave secreta compartilhada**.
 - Esta chave é usada para **criptografia simétrica** dos payloads (dados) do TLS.
- **2. Integridade da Mensagem**
 - O Protocolo Handshake também define (outra) **chave secreta compartilhada**.
 - Esta chave é usada para formar um **MAC (Message Authentication Code)**.
- **Tipos de Conteúdo (Content Types) transportados:**
 - *change_cipher_spec* (Protocolo de Gerenciamento TLS)
 - *alert* (Protocolo de Gerenciamento TLS)
 - *handshake* (Protocolo de Gerenciamento TLS)
 - *application_data* (Dados da Aplicação, ex: HTTP).
- **Obs:** O conteúdo de *application_data* é **opaco** para o TLS (ele não sabe o que é HTTP, apenas o protege).

Protocolo Change Cipher Spec

- Esse protocolo utiliza o Protocolo de Registro (TLS Record Protocol).
- É o **mais simples** de todos os protocolos de gerenciamento do TLS.
- **Estrutura do Protocolo:**
 - Consiste em uma **única mensagem**.
 - Essa mensagem contém apenas **um byte**.
 - O valor desse byte é **1**.
- **Finalidade (Ação Chave):**
 - A sua única finalidade é sinalizar uma transição.
 - A mensagem "Change Cipher Spec" **NÃO** é parte do Handshake, mas sim um sinal *entre* fases.
- **O Que a Mensagem Causa?**
 - Atualiza (ativa) o cipher suite a ser usado nesta conexão a partir deste ponto.

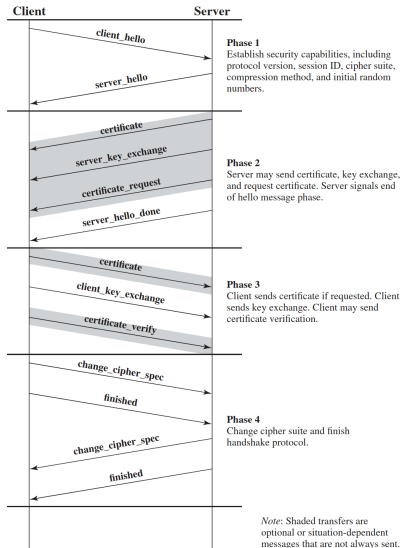
Protocolo de Alerta (Alert Protocol)

- **Finalidade:** Usado para transmitir alertas relacionados ao TLS para o outro lado.
- **Segurança:** Assim como os dados da aplicação, as mensagens de alerta são **comprimidas e criptografadas**.
- **Estrutura da Mensagem (2 bytes):**
 - **Primeiro Byte (Severidade):**
 - 'warning(1)': Um aviso (não fatal).
 - 'fatal(2)': Um erro fatal.
 - **Segundo Byte (Código):**
 - Indica o alerta específico (ex: 'bad_record_mac', 'close_notify').
- **Consequência de Alertas Fatais:**
 - Se o nível for 'fatal', o TLS **termina a conexão imediatamente**.
 - Outras conexões na mesma sessão podem continuar.
 - Nenhuma *nova conexão* pode ser estabelecida nesta sessão.
- **Exemplos de Alertas:**
 - **Exemplo Fatal:** 'incorrect MAC' (MAC incorreto).
 - **Exemplo Não Fatal (Warning):** 'close_notify' (notifica o destinatário que o remetente não enviará mais mensagens nesta conexão).

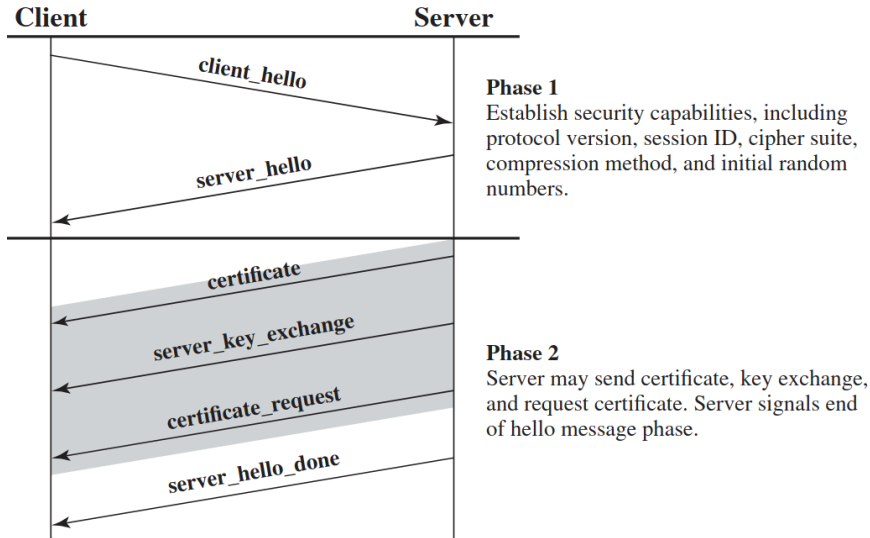
Protocolo Handshake (Handshake Protocol)

- É a parte **mais complexa** do TLS.
- Usado **antes** que quaisquer dados da aplicação sejam transmitidos.
- **Permite que o Servidor e o Cliente:**
 - Autentiquem-se mutuamente (especialmente o servidor ao cliente).
 - Negociem um algoritmo de **criptografia**.
 - Negociem um algoritmo de **MAC** (integridade).
 - Negociem as **chaves criptográficas** a serem usadas para proteger os dados.
- Consiste em uma série de mensagens trocadas entre cliente e servidor, divididas em quatro fases (conforme Figura 22.6).

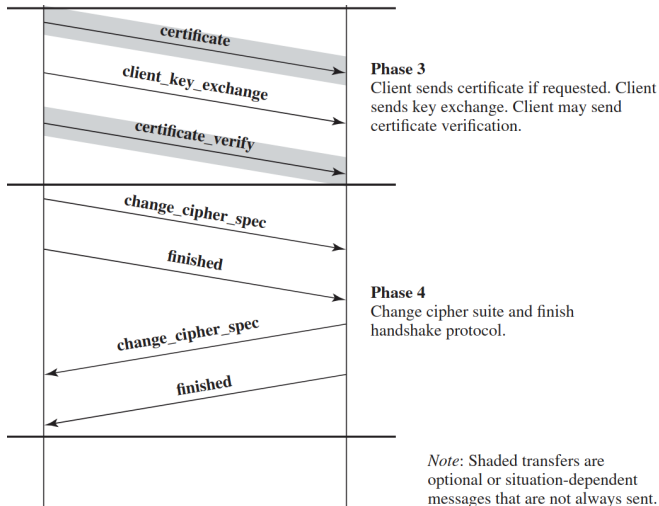
Protocolo Handshake do TLS



Protocolo Handshake do TLS (zoom 1)



Protocolo Handshake do TLS (zoom 2)



2.6 Handshake Protocol Action

As Quatro Fases do Handshake (Visão Geral)

- O protocolo consiste em uma série de mensagens trocadas.
- A troca pode ser vista como tendo quatro fases distintas:
- **Fase 1: Iniciação e Capacidades**
 - Inicia a conexão lógica (`client_hello`, `server_hello`).
 - Estabelece as capacidades de segurança (o que ambos suportam).
- **Fase 2: Autenticação do Servidor**
 - O servidor envia seu certificado e (possivelmente) informações de chave.
 - Termina com `server_done`.
- **Fase 3: Resposta do Cliente**
 - O cliente verifica o servidor e envia suas informações de chave (ex: `ClientKeyExchange`).
- **Fase 4: Finalização**
 - Completa a configuração da conexão segura, ativando os novos parâmetros (`ChangeCipherSpec`, `Finished`).

Handshake - Fase 1 (Início: Client Hello)

- A troca é iniciada pelo **cliente**, que envia uma mensagem `client_hello`.
- **Parâmetros do `client_hello`:**
- **Version:** A **versão mais alta** do TLS que o cliente entende.
- **Random:** Uma estrutura aleatória gerada pelo cliente.
 - Composição: 32-bit timestamp + 28 bytes aleatórios (de um gerador seguro).
 - Usado para prevenir ataques de *replay*.
- **Session ID:**
 - **Valor diferente de zero:** Cliente deseja atualizar parâmetros de uma conexão existente ou criar nova conexão nesta sessão (retomada de sessão).
 - **Valor zero:** Cliente deseja estabelecer uma nova conexão em uma nova sessão.
- **CipherSuite:**
 - Uma **lista** das combinações de algoritmos criptográficos que o cliente suporta.
 - Em **ordem decrescente de preferência**.
 - Cada item define: algoritmo de troca de chave E um CipherSpec (algoritmo de cifra/MAC).
- **Compression method:** Lista de métodos de compressão suportados.

Handshake - Fase 1 (Resposta: Server Hello)

- Após enviar o `client_hello`, o cliente aguarda a resposta.
- O servidor responde com uma mensagem `server_hello`.
- **Conteúdo do `server_hello`:**
 - Contém os **mesmos parâmetros** do `client_hello`.
 - **Porém:** O servidor agora **selecionou UMA opção** de cada lista (as que serão usadas).
 - **Version:** A versão que será usada (escolhida pelo servidor).
 - **Random:** O valor aleatório do **servidor**.
 - **Session ID:** O ID da sessão (se for retomada ou nova).
 - **CipherSuite:** A **única** suíte de cifras (escolhida pelo servidor) que será usada.
 - **Compression method:** O **único** método (escolhido pelo servidor) que será usado.

Handshake - Fase 2 (Autenticação do Servidor)

- Os detalhes desta fase dependem do esquema de criptografia de chave pública (definido no CipherSuite).
- **O que o servidor envia (após o `server_hello`):**
 - **1. Certificado (Certificate):**
 - (Quase sempre) O servidor envia seu certificado digital para o cliente se autenticar.
 - **2. Informação de Chave (ServerKeyExchange):**
 - (Opcional) Pode enviar informações adicionais de chave (ex: parâmetros Diffie-Hellman).
 - **3. Solicitação de Certificado (CertificateRequest):**
 - (Opcional) O servidor pode solicitar um certificado do cliente (para autenticação mútua).
 - **4. Fim da Mensagem (Server Done):**
 - **Sempre obrigatória.**
 - O servidor envia a mensagem `server_done`.
 - Indica o fim do "olá" do servidor e mensagens associadas.
 - Após enviar, o servidor aguarda a resposta do cliente.

Handshake - Fase 3 (Resposta do Cliente)

- Após receber o `server_done` do servidor:
- **1. Verificação (Ações do Cliente):**
 - O cliente deve verificar se o servidor forneceu um **certificado válido** (se foi exigido).
 - O cliente deve checar se os parâmetros do `server_hello` (CipherSuite, etc.) são aceitáveis.
- **2. Envio de Mensagens (Ações do Cliente):**
 - Se tudo estiver satisfatório, o cliente envia uma ou mais mensagens ao servidor.
 - (Ex: `ClientKeyExchange` - contendo o "pre-master secret" criptografado com a chave pública do servidor).
 - (Ex: `CertificateVerify` - se o servidor solicitou um certificado do cliente e o cliente está respondendo).

Handshake - Fase 4 (Finalização - Ativação)

- **1. Ação do Cliente:**

- Envia a mensagem `change_cipher_spec`.
- (Esta mensagem usa o *Change Cipher Spec Protocol*, não o Handshake Protocol).
- Ação: Copia o CipherSpec *pendente* (negociado) para o CipherSpec *atual* (em uso).
- Imediatamente, envia a mensagem `finished`.
- **Importante:** A mensagem `finished` já é enviada usando os **novos** algoritmos, chaves e segredos.
- A `finished` verifica se a troca de chaves e a autenticação foram bem-sucedidas.

- **2. Ação do Servidor (em resposta):**

- Envia sua própria mensagem `change_cipher_spec`.
- Transfere o CipherSpec *pendente* para o atual.
- Envia sua própria mensagem `finished` (também já criptografada).

- **3. Conclusão:**

- Neste ponto, o handshake está completo.
- Cliente e servidor podem começar a trocar dados da camada de aplicação (HTTP) de forma segura.

Protocolo Heartbeat: Definição e Contexto

- **O que é um "Heartbeat"?** (Contexto Geral)
 - Um sinal periódico (hardware ou software) usado para indicar operação normal ou para sincronizar partes de um sistema.
- **O que é um "Protocolo Heartbeat"?**
 - Usado para monitorar a **disponibilidade** de uma entidade de protocolo (verificar se o outro lado está "vivo").
- **No contexto SSL/TLS:**
 - Foi definido em 2012 na **RFC 6250**.
 - Nome oficial: "Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) Heartbeat Extension".

Protocolo Heartbeat: Propósitos

- O heartbeat serve a dois propósitos principais:
- **Propósito 1: "Keep-Alive" (Verificação de Atividade)**
 - Assegura ao remetente que o destinatário ainda está "vivo".
 - É útil mesmo que não tenha havido atividade (envio de dados da aplicação) sobre a conexão TCP subjacente por um tempo.
- **Propósito 2: "Firewall Traversal" (Evitar Ociosidade)**
 - Gera atividade (tráfego) na conexão durante períodos ociosos (idle).
 - **Objetivo:** Evitar o fechamento da conexão por *firewalls* que não toleram conexões ociosas.

Protocolo Heartbeat: Operação e Negociação

- **Posição na Pilha:**
 - Roda "em cima" (on top) do Protocolo de Registro TLS (TLS Record Protocol).
- **Tipos de Mensagem (2):**
 - 'heartbeat_request'
 - 'heartbeat_response'
- **Negociação (Como é ativado?):**
 - O uso do Heartbeat é estabelecido durante a **Fase 1 do Protocolo Handshake**.
 - Cada lado (peer) indica se suporta heartbeats.
 - Se suportado, o lado indica seu modo de operação:
Modo 1 Disposto a *receber requests* e *responder* com responses.
Modo 2 Disposto *apenas a enviar* requests.

Protocolo Heartbeat: Estrutura da Mensagem

- **Fluxo:**
 - Uma 'heartbeat_request' pode ser enviada a qualquer momento.
 - Ao receber um *request*, o receptor deve responder *prontamente* com um 'heartbeat_response'.
- **Conteúdo da Mensagem request:**
 - **Payload (Carga Útil):**
 - Um conteúdo **aleatório**.
 - Tamanho: entre 16 bytes e 64 Kbytes.
 - **Payload Length (Comprimento):** O tamanho do payload.
 - **Padding (Preenchimento):** Um conteúdo aleatório adicional.
- **Requisito da Mensagem response:**
 - Deve incluir uma **cópia exata** do *payload* recebido no *request*.
- **Uso do Padding:**
 - Permite a descoberta de Path MTU (PMTU).
 - (Enviando *requests* com *padding* crescente até que a resposta falhe).

Ataques SSL/TLS: Introdução e Evolução

- Desde a introdução do SSL (1994) e a padronização do TLS, inúmeros ataques foram criados.
- **Ciclo de Ameaça e Resposta:**
 - A aparição de cada ataque necessitou mudanças (contramedidas) no protocolo, nas ferramentas de criptografia ou nas implementações.
- **Categorias Gerais de Ataque:**
 - 1. Ataques ao Protocolo Handshake.
 - 2. Ataques aos protocolos de Registro (Record) e Dados da Aplicação.
 - 3. Ataques à PKI (Infraestrutura de Chave Pública).
 - 4. Outros ataques (ex: DoS).
- **A Realidade dos Protocolos de Internet:**
 - Um protocolo "perfeito" ou implementação "perfeita" nunca é alcançado.
 - A evolução dos protocolos é determinada por um constante "vai-e-vem" (back-and-forth) entre ameaças e contramedidas.

Categoria 1: Ataques ao Protocolo Handshake

- **Objetivo:** Comprometer o Handshake Protocol.
- **Exemplo (1998, [BLEI98]):**
 - Apresentada uma abordagem baseada na exploração da **formatação e implementação** do esquema de criptografia RSA.
- **Evolução do Ataque:**
 - À medida que as contramedidas eram implementadas, o ataque foi **refinado e ajustado** (ex: [BARD12]).
 - **Objetivo do Refinamento:** Não apenas burlar as contramedidas, mas também **acelerar** o tempo do ataque.

Categoria 2: Ataques aos Protocolos de Registro e Aplicação (BEAST)

- Diversas vulnerabilidades foram descobertas nestes protocolos, levando a *patches* (correções).
- **Exemplo 1: BEAST (2011)**
 - (Browser Exploit Against SSL/TLS) - por Thai Duong e Juliano Rizzo [GOOD11].
 - Transformou o que era considerado apenas uma vulnerabilidade **teórica** em um ataque **prático**.
 - **Tipo de Ataque:** *Chosen-Plaintext Attack* (Ataque de Texto Plano Escolhido).
 - (O atacante monta o ataque escolhendo um "palpite" (guess) para o texto plano associado a um texto cifrado conhecido).
 - **Contramedida:** Patches subsequentes foram capazes de bloquear este ataque.

Categoria 2: Ataques (Continuação - CRIME)

- **Exemplo 2: CRIME (2012)**
 - (Compression Ratio Info-leak Made Easy) - pelos mesmos autores do BEAST [GOOD12b].
 - **Como Funciona:** Explora uma vulnerabilidade quando a **compressão de dados** é usada juntamente com o TLS.
 - **Resultado:** Permite ao atacante recuperar o conteúdo de *web cookies*.
 - **Ameaça Principal:** Se usado para recuperar cookies de autenticação secretos, permite ao atacante realizar **sequestro de sessão (session hijacking)**.

Categoria 3: Ataques à PKI (Infraestrutura de Chave Pública)

- **Vulnerabilidade:** A atividade de verificar a **validade de certificados X.509** está sujeita a ataques.
- **Exemplo (2012, [GEOR12]):**
 - Demonstrou que bibliotecas SSL/TLS populares sofriam de implementações **vulneráveis de validação de certificados**.
 - Foram reveladas fraquezas no código-fonte de:
 - OpenSSL
 - GnuTLS
 - JSSE (Java)
 - ApacheHttpClient
 - cURL
 - PHP, Python
 - E aplicações construídas sobre elas.

Categoria 4: Outros Ataques (Ex: DoS)

- Refere-se a ataques que não se encaixam nas categorias anteriores [MEYE13].
- **Exemplo: Ataque de Negação de Serviço (DoS) (2011)**
 - Anunciado pelo grupo hacker "The Hackers Choice" [KUMA11].
 - **Como Funciona:** Cria uma carga de processamento pesada no servidor, **sobrecarregando-o com requisições de handshake SSL/TLS**.
 - (Usando o estabelecimento de novas conexões ou a "renegociação" (renegotiation)).
 - **Por que Funciona (Assimetria):** A maioria da computação (CPU) durante um handshake é feita pelo **servidor**, não pelo cliente.
 - **Resultado:** O servidor é forçado a computar continuamente números aleatórios e chaves, levando à exaustão de recursos (DoS).

Ataque: Heartbleed (2014)

- Uma das vulnerabilidades TLS potencialmente **mais catastróficas** já descobertas.
- **Onde:** Encontrado na popular biblioteca *open-source* **OpenSSL**.
- **O Quê:** Um bug (falha) na implementação do **Protocolo Heartbeat** (RFC 6250).
- **Ponto Crítico:**
 - **NÃO** foi uma falha de design (design flaw) na especificação do TLS ou do Heartbeat.
 - Foi um **erro de programação** (programming mistake) específico da biblioteca OpenSSL.

Heartbleed: A Natureza da Vulnerabilidade

- **Revisão do Heartbeat (Comportamento Esperado):**

- O Cliente envia um `heartbeat_request`.
- O *request* contém um campo **Payload Length** (L) e um **Payload** (P).
- O Servidor deve responder com uma **cópia exata** do Payload (P).

- **A Falha de Programação (O Bug):**

- O software OpenSSL (versão vulnerável) **NÃO CHECAVA** se o tamanho real (bytes) do *payload* recebido era igual ao valor no campo *Payload Length*.

- **O Vetor de Ataque (Exploit):**

- Um adversário envia um `heartbeat_request` malicioso:
- Define o **Payload Length** = 64 KB (o máximo).
- Envia um **Payload** real de apenas 16 bytes (o mínimo).

Heartbleed: O Mecanismo do Vazamento (O Exploit)

- **1. Alocação (No Servidor):**

- O servidor (vulnerável) recebe o *request*.
- Ele lê o **Payload Length (64 KB)** e aloca um *buffer* de memória deste tamanho (64 KB).

- **2. Cópia (O Bug):**

- O servidor copia o *payload* recebido (os **16 bytes** reais) para o início deste buffer.
- **O Problema:** Os 63.9 KB restantes do buffer **não são sobrescritos** (not overwritten).
- Este espaço (quase 64 KB) contém o que quer que estivesse na memória RAM do servidor naquele momento (lixo de memória).

- **3. A Resposta (O Vazamento):**

- O servidor, obedecendo o protocolo, prepara uma *heartbeat_response*.
- Ele lê o **Payload Length (64 KB)** do *request* original e envia de volta **64 KB** de dados do buffer.
- **Resultado:** O atacante recebe seus 16 bytes + 63.9 KB de **dados da memória do servidor**.

Heartbleed: O Impacto Catastrófico

- **Uma "Falha Espetacular":**
 - Ataques repetidos resultam na exposição de grandes quantidades de memória do sistema vulnerável.
- **O que a memória (não sobrescrita) poderia conter?**
 - **Chaves Privadas** (a "joia da coroa" de um servidor SSL).
 - Informações de identificação de usuário.
 - Dados de autenticação (cookies de sessão).
 - Senhas, e outros dados sensíveis.
- **A Tempestade Perfeita:**
 - **Longo Período de Exposição:** O bug não foi descoberto por vários anos.
 - **Ataque Fácil:** O *exploit* é trivial de implementar.
 - **Sem Vestígios:** O ataque não deixa rastros (logs) de que ocorreu.
- **A Escala do Problema:**
 - OpenSSL era a implementação de TLS **mais usada** no mundo.
 - Afetou finanças, bancos, e-mail, redes sociais e governos.
 - Estimativa (na época): Mais de **dois terços (2/3)** dos servidores Web da Internet usavam OpenSSL [GOOD14].